

UNIVERSITÀ DEGLI STUDI DI ROMA TOR VERGATA



ESAME

Performance Modeling of Computer Systems and Networks

TITOLO PROGETTO

*Analisi Prestazionale e Dimensionamento di un'Architettura
Edge-Cloud per Veicoli Connessi*

STUDENTE

Simone Canapini

MATRICOLA

0381825

Anno Accademico 2025/2026

Sommario

1. Introduzione.....	3
1.1 Contesto e Motivazioni.....	3
1.2 Descrizione del Caso di Studio.....	4
1.3 Miglioramenti Attesi.....	5
2. Descrizione del Sistema e Obiettivi.....	6
2.1 Obiettivo 1: Dimensionamento del Nodo.....	6
2.2 Obiettivo 2: Valutazione di un evento anomalo.....	7
3. Modellazione del Sistema.....	9
3.1 Il Modello Concettuale.....	9
3.2 Il Modello di Specifica.....	13
3.3 Il Modello Computazionale.....	17
4. Verifica e Validazione.....	19
4.1 Verifica dell'Implementazione.....	19
4.1.1 Verifica Logica e Tracciamento (Event Tracing).....	19
4.1.2 Verifica delle Condizioni al Contorno (Edge Cases).....	22
4.1.3 Controllo della Conservazione (Flow Balance).....	23
4.1.4 Verifica dell'Impianto Stocastico (RNG e RVG).....	24
4.2 Validazione del Modello.....	26
4.2.1 Analisi del Carico e Stabilità.....	26
4.2.2 Analisi di Sensibilità (Controlli di Coerenza).....	28
4.2.3 Validazione Analitica Globale.....	30
5. Progettazione degli Esperimenti e Risultati.....	31
5.1 Metodologia di Inferenza Statistica.....	31
5.2 Analisi Obiettivo 1: Dimensionamento Edge.....	33
5.2.1 Identificazione del Transitorio.....	33
5.2.2 Risultati e determinazione del numero minimo di server.....	35
5.3 Analisi Obiettivo 2: Congestione Dinamica.....	37
5.3.1 Scenario di Baseline e Starvation.....	37
5.3.2 Scenario Ottimizzato e Analisi del Transitorio.....	40
6. Evoluzione del Modello	43
7. Conclusioni e Sviluppi Futuri.....	53

1. Introduzione

Questo capitolo introduce il progetto di valutazione prestazionale, delineando lo scenario applicativo e le problematiche tecnologiche affrontate. Nelle sezioni seguenti verrà descritto il contesto dei trasporti smart, verranno dettagliate le caratteristiche dell'infrastruttura di rete analizzata e verranno definiti i miglioramenti attesi dall'implementazione delle logiche di controllo proposte.

1.1 Contesto e Motivazioni

Negli ultimi anni, il paradigma delle *Smart City* ha guidato una profonda trasformazione degli ecosistemi urbani, ponendo l'efficienza, la sostenibilità e la sicurezza al centro dell'innovazione tecnologica. Un pilastro fondamentale di questa evoluzione è rappresentato dai sistemi di trasporto intelligente (*Smart Transportation*), all'interno dei quali i veicoli connessi, noti come V2X (*Vehicle-to-Everything*), giocano un ruolo da protagonisti. In questo scenario, i veicoli interagiscono costantemente con l'infrastruttura di rete urbana, generando enormi moli di dati in tempo reale.

Gestire questo flusso continuo di informazioni rappresenta una sfida tecnologica notevole, specialmente considerando la natura critica di molte di queste comunicazioni. In questo contesto, infatti, i dati scambiati non hanno tutti la stessa valenza. Esistono applicazioni *safety-critical*, come l'invio di segnali di emergenza per avvisi di collisione imminente, rilevamento di pedoni o frenate brusche, che richiedono un'elaborazione e una trasmissione immediate (in broadcast ai veicoli vicini) per garantire la sicurezza su strada.

Storicamente, l'aggregazione e l'elaborazione dei dati IoT sono state affidate quasi esclusivamente a infrastrutture Cloud centralizzate. Tuttavia, sebbene il Cloud offra risorse praticamente illimitate per l'archiviazione a lungo termine o per calcoli altamente complessi (come l'addestramento di modelli di machine learning su scala globale), non è adeguato alla gestione in tempo reale del traffico veicolare critico. La distanza fisica e topologica tra i data center centralizzati e i dispositivi finali (i veicoli) introduce infatti latenze di rete imprevedibili e troppo elevate. Per sistemi di sicurezza attiva, dove un ritardo di pochi millisecondi può determinare il verificarsi di un incidente, questo approccio risulta inaccettabile.

Da qui nasce l'esigenza di superare i limiti dell'elaborazione centralizzata, spostando la computazione il più vicino possibile alla sorgente dei dati. Questa necessità ha portato all'introduzione del paradigma *Edge Computing*. Distribuendo nodi computazionali periferici lungo le strade, ad esempio presso le stazioni radio base 5G, il livello Edge si fa carico di elaborare istantaneamente le richieste che esigono una bassissima latenza. Questa decentralizzazione non solo decongestiona la rete verso il Cloud, ma diventa la tecnologia abilitante e indispensabile per garantire i rigorosi tempi di risposta richiesti dai moderni sistemi di trasporto intelligente.

1.2 Descrizione del Caso di Studio

Il sistema analizzato in questo progetto si basa su un'architettura ibrida a due livelli, progettata per operare all'interno di un ambiente di Smart Transportation. Questa architettura è composta da un livello periferico (Edge) e da un livello centrale (Cloud). Il livello Edge è costituito da nodi computazionali distribuiti lungo le infrastrutture stradali e collocati presso le stazioni radio base 5G. Questi nodi hanno il compito di fornire risorse di calcolo di prossimità per soddisfare tutte le richieste che esigono una bassa latenza. Al contrario, il livello Cloud centrale è deputato alla gestione dei compiti computazionali più onerosi e non soggetti a stringenti vincoli temporali, occupandosi principalmente dell'archiviazione dei dati a lungo termine e dell'esecuzione di calcoli complessi, come l'addestramento di modelli di machine learning sull'andamento del traffico globale.

Nell'ambiente simulato, i veicoli connessi producono un flusso di dati eterogeneo, caratterizzato da diverse esigenze in termini di prestazioni e tempi di risposta. Il modello proposto classifica questo traffico in tre categorie principali:

- Segnali di Emergenza (Safety-critical): Comprendono le notifiche relative a eventi critici per la sicurezza, come avvisi di collisione imminente, frenate brusche o il rilevamento di pedoni. Tali richieste necessitano di un'elaborazione istantanea presso il nodo Edge, seguita da una rapida trasmissione in broadcast verso i veicoli limitrofi. Per garantire l'efficacia del servizio, il sistema implementa una coda di priorità ai nodi Edge, assegnando a questi segnali la priorità più elevata. Il vincolo di prestazione per questa categoria è rigoroso: il tempo di risposta all'Edge deve essere inferiore o uguale a 15 ms.

- Navigazione Real-Time: Riguarda le richieste inviate dai veicoli per il ricalcolo del percorso basato sulle condizioni del traffico locale. Anche questa categoria viene elaborata localmente al livello Edge, prevedendo l'eventuale interrogazione di database locali. Le prestazioni richieste sono inferiori rispetto alle emergenze, ma il tempo di servizio per queste elaborazioni presenta un'elevata varianza, essendo strettamente legato alla complessità del percorso richiesto dal singolo utente.
- Telemetria di Routine: Consiste nell'invio periodico di informazioni sullo stato del veicolo, come la velocità media, i consumi di carburante e i parametri operativi del motore. Trattandosi di misurazioni non critiche, questi dati tollerano ritardi di trasmissione elevati. Pertanto, le richieste di telemetria non vengono processate dai server periferici, ma vengono semplicemente instradate dal livello Edge verso l'infrastruttura Cloud per la successiva archiviazione e analisi.

1.3 Miglioramenti Attesi

L'analisi e la simulazione di questa architettura si prefiggono di dimostrare il valore e l'efficacia di una corretta ingegnerizzazione delle risorse computazionali di rete. In primo luogo, ci si aspetta di confermare che un preventivo e adeguato dimensionamento hardware del singolo nodo Edge sia in grado di garantire il rispetto dei vincoli di sicurezza. Nello specifico, la simulazione mira a dimostrare che, sotto un carico di traffico nominale, l'infrastruttura può soddisfare il Service Level Agreement (SLA) imposto per i Segnali di Emergenza, assicurando un tempo di risposta costantemente inferiore o uguale a 15 ms.

Inoltre, il progetto si propone di evidenziare un netto miglioramento della resilienza complessiva del sistema in presenza di eventi anomali, come un incidente stradale che causa un improvviso e massiccio picco temporaneo (*burst*) di segnalazioni. In scenari di forte congestione dinamica, l'architettura introduce una politica di offloading dinamico. Ci si aspetta che questo meccanismo di bilanciamento del carico, in grado di instradare parte delle richieste meno critiche (come la Navigazione Real-Time) verso i nodi Edge dei quartieri adiacenti, risulti determinante. L'attesa è che tale politica si riveli una soluzione efficace per preservare la massima reattività del sistema verso le classi di traffico ad altissima priorità, scongiurando il collasso del nodo locale e garantendo la continuità dei servizi essenziali per la sicurezza stradale.

2. Descrizione del Sistema e Obiettivi

Questo capitolo ha lo scopo di formalizzare il comportamento operativo del nodo Edge, definendone le dinamiche interne, la logica di accodamento e le regole di gestione delle diverse priorità. A partire da questa struttura, vengono definiti i due principali scenari di test che la simulazione andrà a misurare per valutare le prestazioni dell'architettura. Nello specifico, l'analisi si articolerà su due condizioni operative distinte: una prima valutazione a regime stazionario (*steady-state*), fondamentale per dimensionare correttamente l'infrastruttura sotto un carico di traffico nominale, e una successiva analisi in regime transitorio (*transient-state*), necessaria per misurare la reattività e la resilienza del sistema a fronte di una congestione dinamica improvvisa.

2.1 Obiettivo 1: Dimensionamento del Nodo

Il primo esperimento condotto nel progetto si concentra sull'analisi del sistema in condizioni di normale funzionamento continuo, ovvero in una situazione di traffico a regime stazionario. Sotto questo profilo, il problema affrontato è il *Capacity Planning* del nodo Edge locale, modellabile analiticamente come un sistema multi-server. Lo scopo è determinare il numero minimo di server paralleli necessari per gestire in modo efficiente il traffico automobilistico standard, ottimizzando l'infrastruttura in modo da evitare sovradimensionamenti che comporterebbero un inutile spreco di risorse hardware ed energetiche.

Il parametro discriminante per il corretto dimensionamento è rappresentato da un Service Level Agreement (SLA). Nello specifico, i Segnali di Emergenza, ai quali è assegnata la priorità di elaborazione più elevata, devono completare l'attraversamento del nodo Edge in un tempo massimo di 15 ms. Tale metrica di prestazione (tempo di risposta) include sia l'eventuale tempo di attesa speso all'interno della coda di priorità, sia il tempo effettivo di servizio per l'elaborazione della richiesta.

Tuttavia, è necessario considerare le dinamiche reali della rete veicolare: la natura intrinsecamente stocastica degli arrivi dei pacchetti e l'elevata varianza che caratterizza il tempo di servizio delle richieste di Navigazione Real-Time rendono impossibile garantire in termini matematici e deterministici che il 100% dei pacchetti di emergenza venga processato entro la

soglia prestabilita. Di conseguenza, il criterio di successo per il soddisfacimento dello SLA è definito in termini statistici: l'obiettivo di dimensionamento è considerato raggiunto se il numero di server selezionato garantisce che la probabilità di violare il vincolo dei 15 ms per i segnali di emergenza sia inferiore all'1%. Ciò equivale alla progettazione di un ecosistema Edge capace di offrire un'affidabilità del 99% per le applicazioni *safety-critical*.

2.2 Obiettivo 2: Valutazione di un evento anomalo

Per il secondo obiettivo, l'attenzione si sposta da un sistema in condizioni di stabilità a un sistema sottoposto a un forte stress improvviso, richiedendo pertanto un'indagine in regime transitorio. Nello specifico, si intende valutare l'impatto sulle prestazioni di un evento anomalo, ovvero una situazione di congestione dinamica causata da un incidente stradale.

La dinamica dell'evento

Fisicamente, il verificarsi dell'incidente altera il comportamento della rete veicolare, provocando un improvviso e massiccio picco temporaneo (*burst*) degli arrivi. In questa fase di crisi, il tasso di arrivo globale quadruplica repentinamente. Parallelamente, si assiste ad una mutazione nella composizione del traffico, determinata tramite stime euristiche: la porzione di Segnali di Emergenza subisce un'impennata, balzando dal 10% al 35% del totale, mentre le richieste di Navigazione Real-Time, generate dai veicoli alla ricerca di percorsi alternativi per aggirare l'ingorgo, diventano la componente dominante, assorbendo il 50% del traffico totale.

La contromisura: Dynamic Offloading

Per prevenire il collasso dell'infrastruttura locale sotto questo carico eccezionale, il sistema implementa una politica di difesa proattiva basata sul bilanciamento del carico (Offloading). Il nodo Edge monitora costantemente il proprio stato, calcolando dinamicamente il carico di lavoro atteso in base alla saturazione della coda locale. Se l'algoritmo stima che il tempo di attesa per le richieste di Navigazione Real-Time sia destinato a superare la soglia dei 50 ms, scatta il meccanismo di deviazione: i nuovi pacchetti appartenenti a questa categoria vengono instradati, tramite fibra ottica, verso il nodo Edge del quartiere adiacente, che agisce così da risorsa computazionale di supporto.

Metriche di valutazione

Per dichiarare il successo della politica di offloading dinamico e quantificare la resilienza del sistema, l'analisi delle prestazioni si concentrerà sulla misurazione delle seguenti metriche:

- Tenuta del vincolo di latenza: Valutazione del picco massimo del tempo di risposta raggiunto dai Segnali di Emergenza durante l'apice del caos, verificando che il sistema riesca a schermare il traffico critico dal sovraccarico generale.
- Tasso di deviazione (Deflection Rate): Quantificazione del volume di traffico "salvato" dal collasso locale, ovvero la percentuale di richieste di navigazione instradate con successo verso l'infrastruttura adiacente.
- Tempo di recupero (Recovery Time): Misurazione dell'intervallo temporale necessario al sistema per smaltire integralmente l'accumulo anomalo nelle code e ripristinare le normali condizioni operative una volta cessato l'evento scatenante.
- Confronto con la Baseline: Analisi comparativa rispetto a uno scenario "peggiore" in cui la politica di offloading è disattivata. Questo confronto servirà a dimostrare l'inevitabile collasso del nodo locale e a evidenziare il fenomeno della starvation a danno delle classi a bassa priorità, come la Telemetria di Routine.

3. Modellazione del Sistema

In questo capitolo viene dettagliato il processo di astrazione del sistema reale, seguendo il paradigma standard della simulazione a eventi discreti. Il percorso di modellazione si articola in tre fasi: dalla definizione logica e strutturale dei componenti (Modello Concettuale), passando per la formalizzazione logico-matematica delle code e del traffico (Modello di Specifica), fino ad arrivare alla progettazione dell'architettura software necessaria per l'implementazione (Modello Computazionale).

3.1 Il Modello Concettuale

Il sistema reale, costituito dal nodo Edge per la gestione del traffico veicolare V2X, viene astratto in un modello a code con classi di priorità e logica di bilanciamento dinamico del carico.

Confini del Sistema

Il livello di dettaglio del modello ha lo scopo catturare le dinamiche di contesa delle risorse computazionali e di instradamento, senza scendere a un livello implementativo troppo basso. Per questo motivo, il modello si concentra esclusivamente sulle prestazioni del singolo nodo Edge locale, escludendo esplicitamente il livello fisico della rete, come le interferenze radio del segnale 5G, la perdita di pacchetti per fading o i protocolli MAC. Inoltre, i nodi esterni all'infrastruttura locale, ovvero il Cloud centrale e i nodi Edge adiacenti verso cui viene effettuato l'offloading, sono considerati come dei veri e propri "pozzi" (*sinks*) esterni. Si assume che i data center Cloud possiedano risorse virtualmente infinite, motivo per cui lo stato delle loro code interne viene ignorato. Allo stesso modo, per il nodo Edge adiacente si assume una capacità sufficiente ad assorbire il traffico transitorio, ignorandone le variabili di stato interne. Di conseguenza, quando un pacchetto di navigazione viene deviato o un dato di telemetria viene instradato, esso esce definitivamente dal sistema simulato e non se ne traccia più il destino logico o computazionale.

Entità e Attributi

Nel paradigma della simulazione a eventi discreti, le entità rappresentano gli oggetti temporanei che attraversano il sistema. Nel caso di studio, le entità coincidono con le richieste (o *job*) generate dai veicoli connessi. Ogni entità è caratterizzata da attributi specifici, il più importante

dei quali è la *Classe di Traffico* (Emergenza, Navigazione o Telemetria), assegnata al momento della generazione del job. Questo attributo è fondamentale poiché determina il comportamento logico dell'entità all'interno del nodo: definisce in quale coda verrà posizionata e la specifica tipologia (e durata) del servizio computazionale richiesto. Un altro attributo implicito ma informativo ai fini della misurazione delle prestazioni è il *Timestamp* (marca temporale di arrivo); esso è strettamente necessario per calcolare il tempo di permanenza totale (latenza) alla fine dell'elaborazione e verificare così il rispetto dei vincoli imposti.

Risorse del Sistema

Le risorse fisiche e logiche volte a smaltire le entità in arrivo si articolano in server e code:

- I Server: Il nodo Edge principale è modellato come una singola stazione di servizio dotata di c server identici disposti in parallelo, i quali rappresentano i core computazionali del sistema. Da un punto di vista logico, ogni singolo server può trovarsi in due soli stati mutuamente esclusivi: *Idle* (Libero) o *Busy* (Occupato). Il numero complessivo di server occupati in un dato istante t definisce la variabile di stato $S_{busy}(t)$.
- Le Code: Per gestire le diverse categorie di traffico in questo sistema multiclasse, sono previste tre code logiche distinte (una per ciascuna classe). Tali code sono modellate come buffer di capacità virtualmente infinita per evitare scarti per saturazione della memoria. All'interno di ogni singola coda vige una disciplina FIFO (First-In-First-Out), ma l'estrazione complessiva da parte dei server è regolata da una politica di accodamento a priorità non-preemptive tra le classi.

L'architettura logica e la topologia del sistema simulato sono schematizzate in *Figura 1*. Come si evince dal diagramma, i confini del sistema racchiudono esclusivamente le dinamiche del nodo locale, trattando il traffico deviato verso il nodo adiacente come un flusso in uscita verso un sink a capacità infinita.

Stato del Sistema

Lo stato di un sistema a eventi discreti è definito come l'insieme minimo di variabili necessarie per descrivere compiutamente la sua condizione operativa e ciò che accade in un qualsiasi istante temporale t . Nel modello proposto, le variabili logiche fondamentali che compongono e tracciano lo stato del sistema sono:

- Stato dei server: La variabile $S_{busy}(t)$, che indica il numero esatto di server attualmente occupati nel nodo Edge principale, assumendo valori compresi tra 0 e c (il numero totale di core).
- Stato delle code: Le variabili $N_E(t)$, $N_N(t)$ e $N_T(t)$, che rappresentano rispettivamente il numero di pacchetti appartenenti alle classi di Emergenza, Navigazione e Telemetria attualmente in attesa nelle rispettive code. Queste variabili, che cambiano valore a ogni arrivo o partenza, permettono di quantificare il livello di congestione per ogni specifica classe di priorità.

Eventi

Nella simulazione, un evento è definito come un accadimento logico e istantaneo in grado di alterare lo stato del sistema aggiornandone le variabili. L'intera evoluzione dinamica del nodo Edge è guidata dal susseguirsi di due sole tipologie di eventi logici:

- L'Evento di Arrivo: Si verifica quando un nuovo pacchetto entra nel nodo, innescando una precisa sequenza decisionale e un potenziale aumento dei job nel sistema. Appena generato l'evento, il pacchetto viene smistato stocasticamente in una delle tre classi tramite le probabilità $p_E = 0.10$, $p_N = 0.30$ e $p_T = 0.60$, che rappresentano la composizione nominale del traffico cittadino. Se il pacchetto appartiene alla classe Navigazione, il sistema verifica le condizioni di congestione per un eventuale *Offloading*. Il router valuta il livello di congestione calcolando il tempo di attesa stimato (Expected Workload), una metrica basata sul numero di pacchetti in coda pesati per i rispettivi tempi medi di servizio. Se tale stima supera una soglia limite prefissata, il pacchetto di navigazione viene intercettato e instradato verso un nodo adiacente, uscendo dal sistema. Se, invece, il pacchetto non viene deviato (o se appartiene a classi che non prevedono offloading), viene assegnato direttamente a un server libero, qualora disponibile. In caso contrario (tutti i server S_{busy} sono occupati), il pacchetto viene inserito nella sua rispettiva coda di attesa. Come ultima operazione, la logica dell'evento programma la schedulazione temporale del pacchetto in arrivo globale successivo, la cui classe verrà determinata stocasticamente al suo ingresso.
- L'Evento di Partenza (Completamento di Servizio): Si manifesta nel momento in cui un server termina di processare un pacchetto. L'entità lascia definitivamente il sistema, consentendo al simulatore di calcolare e registrare il suo tempo di risposta totale. Sotto il profilo logico, l'evento libera un server e innesca un meccanismo di ripescaggio basato

su uno scheduling a priorità. Il server appena liberato ispeziona le code e preleva il primo pacchetto disponibile per l'elaborazione, rispettando il seguente ordine di importanza: se presente, preleverà un'Emergenza; altrimenti una Navigazione; altrimenti una Telemetria. Se, e solo se, tutte e tre le code risultano vuote, il server transiterà nello stato logico *Idle*.

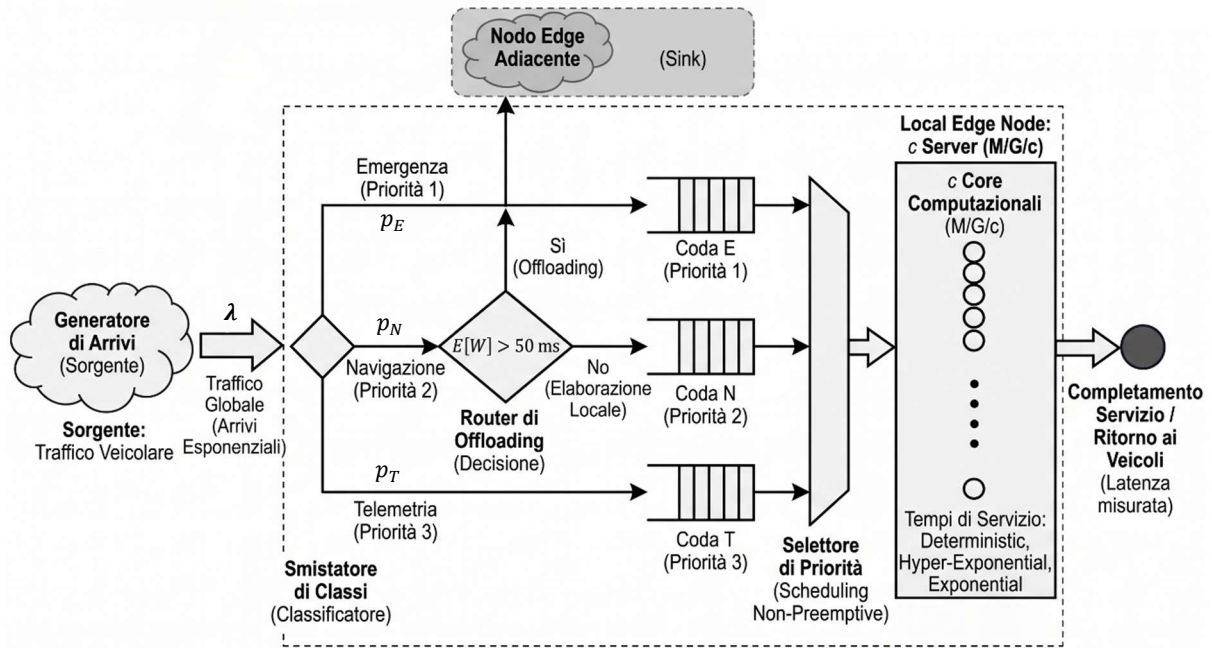


Figura 1: **Modello Concettuale della rete di code del nodo Edge.** Il diagramma illustra la classificazione del traffico in arrivo, la logica di offloading basata sull'Expected Workload e l'accodamento a priorità stretta verso i core computazionali.

3.2 Il Modello di Specifica

In assenza di tracce di traffico reali (dataset empirici completi) specifiche per l'infrastruttura in esame, il traffico in ingresso e i tempi di elaborazione del nodo Edge sono stati modellati mediante l'impiego di variabili casuali note. Tali distribuzioni stocastiche non sono state selezionate in modo arbitrario, ma sono state scelte in modo da riflettere fedelmente la natura fisica del fenomeno reale simulato.

Modelli di Input (Scelta delle distribuzioni stocastiche)

Il processo di modellazione degli input si divide nello studio del comportamento degli arrivi (il carico di lavoro imposto dalla rete) e nell'analisi delle tempistiche di elaborazione richieste all'hardware.

Modello degli Arrivi (Generazione del Traffico)

Nell'ecosistema veicolare, il traffico in ingresso all'Edge è generato in maniera concorrente da una moltitudine di utenti (i veicoli) del tutto indipendenti tra loro. In base al Teorema di Palm-Khintchine, la sovrapposizione di un numero sufficientemente ampio di flussi di traffico indipendenti tende asintoticamente a comportarsi come un Processo di Poisson. Sulla base di questo fondamento teorico, in condizioni di traffico nominale (Obiettivo 1), i tempi di inter-arrivo dei pacchetti per tutte le classi di priorità sono stati modellati sfruttando una Distribuzione Esponenziale. Per quanto riguarda invece l'analisi del transitorio (Obiettivo 2), il modello prevede la simulazione dell'evento anomalo (l'incidente stradale) alterando esplicitamente il processo di arrivo. All'interno del simulatore, questo si traduce nell'applicazione di un brusco e improvviso cambiamento al parametro del tasso globale degli arrivi (λ), accompagnato da una ridistribuzione delle percentuali di traffico, al fine di riprodurre fedelmente il repentino burst di pacchetti sulla rete.

Modelli dei Tempi di Servizio (Service Demands)

Le tre classi di traffico identificate non si differenziano solo per la priorità assegnatagli, ma anche per lo sforzo computazionale richiesto ai server. Utilizzare un'unica distribuzione statistica per tutte le richieste produrrebbe una stima falsata delle prestazioni. La scelta delle distribuzioni per i tempi di servizio si basa sull'analisi fisica delle operazioni:

- Segnali di Emergenza (Classe E): Questi pacchetti (ad esempio il *trigger* di una frenata autonoma d'emergenza) sono strutturalmente molto piccoli, nell'ordine di pochi byte, e richiedono ai server un'elaborazione del tutto standardizzata e rapidissima. Non presentando variabilità intrinseca, il tempo di servizio per questa classe è modellato mediante una distribuzione a bassissima varianza: la distribuzione Deterministica (Costante), caratterizzata da un valore atteso volutamente basso (ad esempio, $D_E = 5$ ms).
- Navigazione Real-Time (Classe N): L'elaborazione per il ricalcolo di un percorso presenta una forte varianza. Il tempo computazionale cambia radicalmente se l'utente richiede le indicazioni per il ristorante nel quartiere vicino, rispetto alla pianificazione di un lungo tragitto autostradale verso un'altra città. Per catturare questa variabilità (Coefficiente di Variazione $CV > 1$), il modello impiega una distribuzione Iperesponenziale a due fasi, con un tempo di servizio medio globale atteso di 30 ms. Nello specifico, la simulazione divide il traffico stocasticamente in due flussi: l'80% delle richieste rappresenta calcoli semplici e veloci (media 10 ms), mentre il restante 20% simula calcoli complessi e lenti (media 110 ms).
- Telemetria (Classe T): Questo traffico consiste nel normale inoltro e pre-processamento di dati strutturati destinati allo storage sul Cloud centrale. Trattandosi di operazioni di routine e non complesse, il tempo di servizio per questa classe è modellato utilizzando la distribuzione Esponenziale (es. $D_T = 10$ ms).

Per la modellazione dei tempi di servizio di Navigazione e Telemetria, si è assunto l'utilizzo di distribuzioni non truncate, ipotizzando l'assenza di meccanismi di timeout hardware sui server. Nella realtà fisica, tali tempi sarebbero soggetti a un limite superiore per evitare latenze di elaborazione tendenti all'infinito; tuttavia, ai fini di questo studio, l'incidenza statistica della coda destra della distribuzione è stata ritenuta trascurabile.

Parametrizzazione

Per consentire l'esecuzione pratica della simulazione e la verifica del vincolo di 15 ms, è stato definito uno scenario di base che assegna valori numerici concreti alle distribuzioni stocastiche delineate. Nello specifico, la composizione del traffico è stata ripartita secondo le seguenti frazioni: 10% per i segnali di Emergenza, 30% per la Navigazione Real-Time e 60% per la Telemetria di Routine. Il tasso di arrivo globale al nodo Edge in condizioni nominali è stato fissato a un valore di riferimento pari a $\lambda = 500$ richieste al secondo.

Per quanto concerne la gestione dinamica del sovraccarico, la decisione di innescare l'offloading si basa su una formula che calcola il Carico di Lavoro Atteso (*Expected Workload*, $E[W]$). Questa equazione pondera i tempi di servizio medi delle richieste attualmente presenti in coda rispetto alla capacità di smaltimento parallela fornita dal numero totale di server c . La formula è formalizzata matematicamente come segue:

$$E[W] = \frac{N_E(t) \cdot 5.0 + N_N(t) \cdot 30.0}{c}$$

Tale equazione restituisce il tempo di attesa stimato all'interno del nodo. Se tale stima supera la soglia limite, il sistema provvede a deviare la richiesta.

Metodo Sistematico per l'Evoluzione Temporale

Per simulare l'evoluzione temporale delle variabili di stato (come il numero di richieste nelle code e lo stato di occupazione dei server), il simulatore adotta il paradigma della Simulazione a Eventi Discreti (Discrete Event Simulation - DES). La spina dorsale logica del sistema si fonda sul meccanismo del *Next-Event Time Advance*: il tempo non scorre in modo continuo, bensì compie dei "salti" da un evento significativo all'altro, saltando i periodi di inattività intermedia. Tutta l'evoluzione è orchestrata da una Lista degli Eventi Futuri (*Event List*).

Le dinamiche interne sono regolate da due algoritmi principali, di seguito descritti in pseudocodice, che formalizzano le reazioni del sistema a ogni evento:

Algoritmo dell'Evento di Arrivo (Arrival Event): Questo evento gestisce la generazione del traffico, il controllo delle soglie e l'occupazione delle risorse.

1. Genera il tempo del prossimo arrivo globale e inseriscilo nella *Event List*.
2. Determina stocasticamente la classe del job (E, N, T) in base alla ripartizione del traffico.
3. SE (la Classe è Navigazione) E ($E[W] > 50.0$ ms):
 - Devia il pacchetto (attiva l'Offloading) e termina l'evento.
4. SE (è disponibile un server libero, ovvero $S_{busy} < c$):
 - Occupalo ($S_{busy} = S_{busy} + 1$).
 - Genera il tempo di servizio per la specifica classe.
 - Schedula l'Evento di Partenza corrispondente nella *Event List*.

5. ALTRIMENTI (tutti i server sono occupati):
 - Inserisci il job nella rispettiva coda di priorità (incrementando la variabile N_E , N_N o N_T).

Algoritmo dell'Evento di Partenza (Departure Event): Questo evento si attiva quando un server si libera, occupandosi di calcolare le latenze e smaltire le code rispettando le priorità.

1. Calcola e registra la latenza totale del job appena concluso.
2. SE ($N_E > 0$, ci sono Emergenze in coda):
 - Preleva il job ($N_E = N_E - 1$), genera il tempo di servizio e schedula una nuova Partenza.
3. ALTRIMENTI SE ($N_N > 0$, ci sono richieste di Navigazione in coda):
 - Preleva il job ($N_N = N_N - 1$), genera il tempo di servizio e schedula una nuova Partenza.
4. ALTRIMENTI SE ($N_T > 0$, ci sono dati di Telemetria in coda):
 - Preleva il job ($N_T = N_T - 1$), genera il tempo di servizio e schedula una nuova Partenza.
5. ALTRIMENTI (tutte le code logiche sono vuote):
 - Il server diviene inattivo ($S_{busy} = S_{busy} - 1$).

3.3 Il Modello Computazionale

Il modello logico-matematico definito nelle sezioni precedenti è stato tradotto in un applicativo software custom, basato sulle fondamenta teoriche di una libreria di simulazione standard. Nello specifico, l'implementazione in linguaggio C è partita dall'estensione dell'architettura originale del simulatore per code multi-server msq.c di Leemis e Park. Il codice è stato ampliato per supportare traffico multiclasse, code di priorità (Non-Preemptive) e logiche di instradamento dinamico (Offloading).

Architettura del Simulatore

Il motore computazionale si basa sul paradigma Next-Event Time Advance. L'orologio di simulazione (variabile `t.current`) non avanza in modo lineare, ma salta direttamente all'istante dell'evento imminente più prossimo.

- **Struttura degli eventi:** La lista degli eventi futuri è gestita tramite una struttura dati che traccia il tempo del prossimo arrivo (`t.next`) e un array contenente i tempi di completamento previsti per ciascuno dei `c` server (`event[i].t`).
- **Ad ogni ciclo del loop di simulazione** (funzione `NextEvent(event)`), il programma individua il tempo minimo tra il prossimo arrivo e le partenze programmate, avanzando l'orologio a tale istante ed eseguendo la routine associata (arrivo o partenza).

Ingegnerizzazione delle Logiche di Coda

Per gestire le tre classi di traffico (Emergenza, Navigazione, Telemetria) e la politica di instradamento, la struttura dati della coda singola di msq.c è stata sostituita da tre contatori di stato e da logiche decisionali dedicate:

- Generazione Multiclasse: Esiste un unico evento di arrivo globale. Al momento dell'arrivo, il programma seleziona un flusso casuale dedicato (es. `SelectStream(3)`) per garantire l'isolamento degli stream casuali, poi invoca il generatore di numeri casuali uniformi `Random()` (dalla libreria `rngs.c`). In base a soglie di probabilità predefinite (es. <0.10 per Emergenza), la richiesta viene classificata. Le soglie di classificazione vengono alterate a runtime durante l'iniezione dell'evento di congestione (Incidente).
- Logica di Offloading Dinamico (Expected Workload): L'equazione dell'Expected Workload è stata implementata calcolando dinamicamente la variabile `expected_wait`. Il costrutto condizionale verifica se tale variabile supera il valore 50.0: in caso

affermativo, il job bypassa l'inserimento in memoria e incrementa direttamente la variabile di deviazione offloaded_N.

- Disciplina di Priorità (Non-Preemptive): L'algoritmo di accodamento è stato ingegnerizzato all'interno della routine di Partenza tramite una catena condizionale *if-then-else* gerarchica. Il sistema ispeziona le variabili di stato queue_E, queue_N e queue_T in quest'ordine esatto, garantendo che i job a bassa priorità vengano prelevati solo se i contatori delle code superiori sono strettamente a zero.

Sistemi di Misurazione e Tracciamento

Per valutare gli indici di prestazione necessari agli obiettivi dello studio, il modello raccoglie statistiche continue (Time-Integrated Variables). L'impiego di questo approccio garantisce una misurazione esatta della lunghezza media delle code all'interno di un ambiente a tempo continuo.

- Aree sotto la curva (Integrazione temporale): Sono state definite specifiche variabili continue di accumulo (es. area_qE, area_qN) che calcolano l'integrale nel tempo del numero di richieste in coda, aggiornandosi a ogni evento tramite il prodotto tra lo stato attuale e il delta temporale ($dt * queue_E$). Parallelamente, la struttura `sum[s].service` accumula il tempo totale in cui ciascun server rimane attivo. Dividendo queste "aree" per il tempo totale di simulazione, il programma è in grado di calcolare, rispettivamente, le lunghezze medie delle code e il tasso di utilizzazione (ρ) dei server.
- Tempi di permanenza e Violazioni: Per verificare il vincolo critico dei 15 ms sull'Emergenza, il programma traccia la marca temporale di ingresso assegnandola direttamente al server se questo è libero, oppure salvandola in un buffer circolare ad array se la richiesta deve attendere in coda. All'evento di completamento del servizio, il sistema calcola la latenza totale come differenza tra il tempo attuale di simulazione e la marca temporale di ingresso. Se tale differenza risulta maggiore di 15 ms, il programma incrementa direttamente un contatore di violazioni, restituendo al termine dell'esecuzione la probabilità deterministica di violazione dello SLA.
- Tasso di Offloading: Per l'Obiettivo 2, si procede al calcolo del rapporto `offloaded_jobs / total_nav_arrivals` per quantificare l'efficacia dell'algoritmo di deviazione dinamica durante gli scenari di picco anomalo del traffico.

4. Verifica e Validazione

Questo capitolo è dedicato all'accertamento della correttezza e dell'affidabilità del simulatore sviluppato, passaggi fondamentali prima di procedere all'estrazione e all'analisi dei risultati. Il processo di controllo è stato diviso in due fasi distinte. Da un lato, la Verifica dell'Implementazione, volta ad assicurare che il modello concettuale e di specifica sia stato tradotto nel codice sorgente in modo esatto, garantendo l'assenza di errori di programmazione o stalli logici (rispondendo alla domanda: "*il modello è stato costruito bene?*"). Dall'altro, la Validazione del Modello, necessaria per dimostrare che il comportamento stocastico del sistema simulato sia matematicamente solido e coerente con le dinamiche reali attese (rispondendo alla domanda: "*è stato costruito il modello giusto?*").

4.1 Verifica dell'Implementazione

Prima di procedere all'estrazione delle metriche prestazionali, il codice sorgente è stato sottoposto a quattro tecniche di software testing specifiche per i simulatori a eventi discreti. L'obiettivo di questa fase propedeutica è garantire la stabilità dell'applicazione ed escludere la presenza di bug nell'avanzamento del tempo o errori di instradamento nelle dinamiche di accodamento.

4.1.1 Verifica Logica e Tracciamento (Event Tracing)

In questa prima sottosezione, si è verificata la correttezza della sequenza decisionale legata agli eventi, assicurandosi che i blocchi condizionali (logiche if-else) si innescassero esattamente negli istanti previsti dall'architettura concettuale.

- Il Test Pilota (Trace Generation): Per eseguire questa validazione, è stata condotta un'esecuzione impostando il simulatore con un tempo di fine simulazione volutamente molto breve, pari a $STOP = 100.0$. Parallelamente, sono state inserite istruzioni di tracciamento (printf) in punti nevralgici del codice (generazione della classe, offloading e partenza) per registrare lo stato delle variabili istante per istante (i risultati aggregati dell'esecuzione sono riassunti nella *Tabella 1*).
- L'Analisi della Traccia: Leggendo l'output testuale stampato a terminale, è stato possibile verificare riga per riga la corretta evoluzione del sistema. L'analisi ha

confermato la correttezza della disciplina di priorità adottata: si evince chiaramente (ad esempio nel blocco di eventi attorno a $t=42$) come i job di Emergenza scavalchino immediatamente le code a bassa priorità non appena si rende disponibile un server. Inoltre, la riduzione capacitiva a soli 2 server ha permesso di validare matematicamente l'algoritmo di deviazione: il sistema ha saturato rapidamente le risorse, facendo scattare la logica basata sull'Expected Workload che ha deviato con successo 4 richieste di Navigazione, portando il tasso di offloading dinamico al 25.00%.

Analisi Visiva (Step-Function)

Per consolidare queste conclusioni su una scala temporale più ampia, è stata prodotta un'analisi visiva delle variabili di stato (*Figura 2*). Questo grafico dimostra visivamente tre certezze logiche fondamentali per la tenuta dell'esperimento:

- Limiti di Risorse (Nessun over-provisioning): La curva relativa ai Server Occupati (linea tratteggiata verde) cresce e si stabilizza rapidamente al limite massimo prefissato, confermando visivamente l'allocazione a 2 server. Il sistema non supera mai questo vincolo capacitivo logico, dimostrando che non vi è alcun *over-provisioning* irregolare.
- Efficacia dell'Offloading: L'azione dell'algoritmo è chiaramente visibile. Nonostante la forte congestione e l'impennata del traffico totale (linea nera), la logica di offloading interviene tempestivamente per limitare i picchi, mantenendo la Coda Navigazione (linea blu) estremamente contenuta (con un accumulo massimo inferiore a 5 job).
- Fenomeno della Starvation: Come atteso dall'utilizzo di priorità, unita a un sistema sotto-dimensionato (2 server), la coda di Telemetria subisce un evidente stato di starvation. Il grafico mostra in modo inequivocabile la Coda Telemetria (linea arancione) esplodere accumulando un picco massimo di 33 job. Essa cede costantemente il passo alle classi critiche finché gli arrivi di Navigazione ed Emergenza non si esauriscono del tutto (dopo $t=110$). Solo a partire da quel momento i server possono dedicarsi allo smaltimento della coda di Telemetria in modo lineare fino al completo svuotamento in fase di chiusura.

Tabella 1: Sintesi delle metriche analizzate (Test con $c = 2$, STOP=100)

Metrica di Sistema	Valore Totale	Ripartizione per Classe
Lunghezza Media Code	-	Coda E: 0.15 Coda N: 0.64 Coda T: 20.18
Offloading Dinamico	4 job deviati	Tasso di Offloading: 25.00% (sulla classe N)

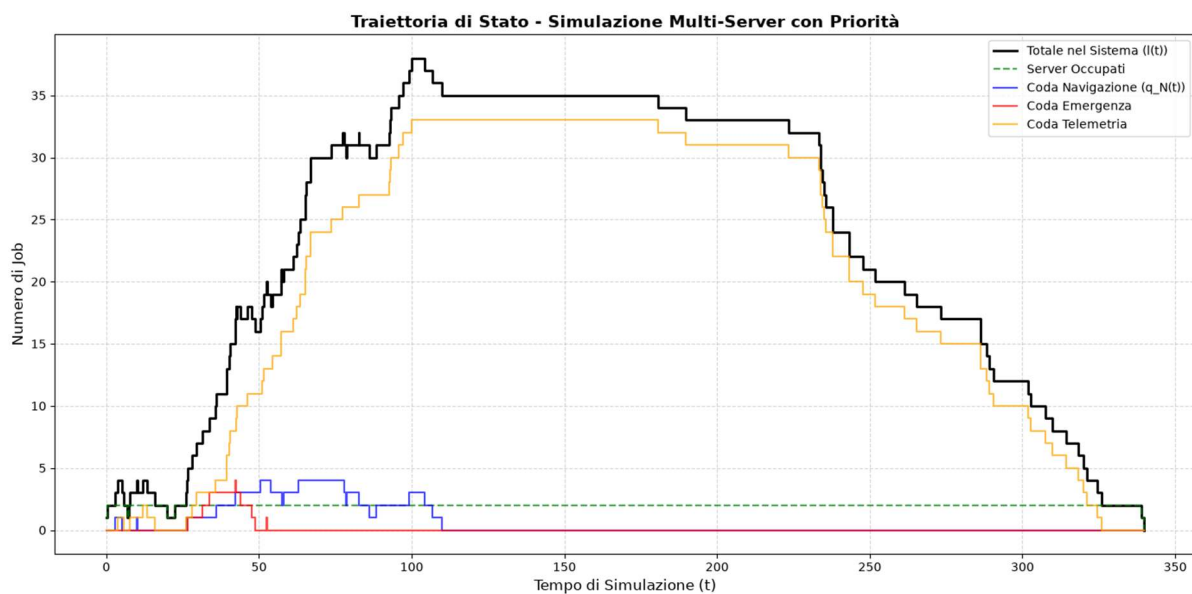


Figura 2: Traiettorie di stato del sistema simulato (Test con capacità ridotta a $c=2$). Il grafico illustra l'evoluzione temporale del numero totale di job e l'occupazione limite dei server (linea tratteggiata verde). È visibile l'efficacia dell'algoritmo di Expected Workload nel contenere i picchi della Coda di Navigazione (linea blu), e il conseguente fenomeno di starvation subito dalla Coda di Telemetria (linea arancione) a causa della disciplina di priorità.

4.1.2 Verifica delle Condizioni al Contorno (Edge Cases)

In questa sottosezione viene dimostrata l'affidabilità del motore di simulazione, confermando che il meccanismo di scorrimento del tempo (*Next-Event Time Advance*) è strutturalmente corretto.

- Il Test dei Tempi Nulli: Per validare la robustezza della lista degli eventi, è stato condotto un esperimento volutamente contro-intuitivo. Il codice sorgente è stato temporaneamente alterato imponendo che tutti i tempi di elaborazione dei server fossero istantanei. Matematicamente, questo si traduce nell'impostare il tempo di servizio atteso nullo, ponendo $E[S_i] = 0.0$ per tutte e tre le classi di traffico.
- I Risultati del Test: L'esito della simulazione (riassunto nella *Tabella 2*) ha confermato appieno le aspettative teoriche. Ponendo $E[S_i] = 0.0$, il sistema ha elaborato un carico massiccio di circa 50.000 eventi in ingresso. Nonostante l'imponente volume di arrivi, le lunghezze medie delle code registrate a fine simulazione sono risultate rigorosamente pari a 0.00 per ogni singola classe. Poiché le code non si sono mai riempite, l'algoritmo di stima del carico non ha mai superato la soglia critica, mantenendo il tasso di deviazione (offloading) esattamente allo 0%.
- Conclusione: Questo risultato certifica matematicamente l'assenza di bug nella gestione degli eventi e l'impossibilità che si verifichino ritardi spuri o "accodamenti fantasma". Il superamento del test estremo conferma la perfetta implementazione del *Next-Event Time Advance* e la correttezza del calcolo degli integrali delle aree (Time-Integrated Variables) utilizzati per l'estrazione delle statistiche.

Tabella 2: Risultati del Test sulle Condizioni al Contorno (c=4, STOP=100.00)

Metrica di Sistema	Valore Rilevato	Note
Arrivi Totali	50012	Volume massiccio per stressare l'Event List.
Lunghezza Media Code	0.00 (per tutte le classi)	Conferma assenza di ritardi spuri.
Offloading Dinamico	0.00% (0 job devianti)	Soglia di attesa mai raggiunta.

4.1.3 Controllo della Conservazione (Flow Balance)

Questa sottosezione fornisce la prova che l'applicativo software non distrugge né duplica accidentalmente i dati durante le operazioni di simulazione.

- La Legge di Bilancio: Al termine dell'esecuzione, all'interno del codice C è stato implementato un controllo logico per verificare il rispetto formale dell'equazione di conservazione del flusso:

$$N_{entrati} = N_{serviti} + N_{deviati} + N_{rimanenti}$$

- La Prova Empirica: I risultati estratti dal test (riportati in *Tabella 3*) confermano la correttezza della logica implementata. A fronte di 50012 pacchetti in ingresso generati dal sistema, la somma esatta dei pacchetti regolarmente processati dai server (46413), dei pacchetti deviati verso il nodo adiacente tramite l'offloading (3599) e delle entità rimaste nel sistema a fine ciclo (0, grazie al completo svuotamento finale) combacia perfettamente.
- Conclusione: Il superamento con successo di questo test esclude la presenza di *memory leaks* o la distruzione accidentale di entità durante le fasi di smistamento, accodamento e prelievo tra le code di priorità.

Tabella 3: Verifica dell'Equazione di Conservazione del Flusso (c=4, STOP=100.000)

Parametro dell'Equazione	Valore Registrato	Descrizione
Entità Entrate ($N_{entrati}$)	50012	Totale degli arrivi generati nel sistema.
Entità Servite ($N_{serviti}$)	46413	Job regolarmente elaborati dai server.
Entità Deviate ($N_{deviati}$)	3599	Job di Navigazione intercettati dall'offloading (24.02%).
Entità Rimanenti ($N_{rimanenti}$)	0	Job residui (azzerati dal ciclo di svuotamento finale).
Esito del Bilancio	50012 = 50012	Test Superato (Nessuna perdita di entità).

4.1.4 Verifica dell'Impianto Stocastico (RNG e RVG)

L'ultima fase di verifica dimostra l'affidabilità del generatore di numeri casuali (RNG) e dei relativi trasformatori (RVG), assicurando l'assenza di bias statistici durante la simulazione.

- Test di Indipendenza Visiva (RNG): Le estrazioni base prodotte dal generatore sono state campionate isolando uno stream dedicato. La proiezione visiva delle coppie adiacenti nel grafico a dispersione (*Figura 3*) evidenzia una distribuzione perfettamente omogenea all'interno del piano bidimensionale. L'assenza totale di pattern geometrici (effetto reticolo) certifica l'indipendenza statistica delle sequenze casuali.
- Segmentazione del Traffico Multiclasse: L'uniformità del generatore ha garantito un corretto bilanciamento probabilistico. A fronte di uno stress-test di 501853 arrivi, l'algoritmo di diramazione ha smistato il traffico in stretta aderenza alle probabilità teoriche del Modello di Specifica: 9.98% per l'Emergenza, 30.06% per la Navigazione e 59.97% per la Telemetria.
- Validazione della Media Campionaria (RVG): Test effettuato sulla trasformazione Iperesponenziale dedicata al calcolo della Navigazione. Su un volume di 150432 estrazioni misurate, il trasformatore ha restituito una media campionaria di 30.03 ms, scostandosi dal valore atteso teorico di 30.00 ms con un errore relativo pari allo 0.103%, generando perciò risultati temporalmente accurati.

I risultati aggregati dell'analisi stocastica, inclusi i controlli incrociati di conformità per i tempi di interarrivo e servizio totali, sono riassunti nella *Tabella 4*.

Tabella 4: Verifica dell'Impianto Stocastico (c=8, STOP=1.000.000)

Parametro Stocastico	Valore Atteso (Teorico)	Valore Rilevato (Empirico)	Scostamento / Errore Relativo
Probabilità Smistamento (Emergenza)	10.00 %	9.98 %	- 0.02 %
Probabilità Smistamento (Navigazione)	30.00 %	30.06 %	+ 0.06 %
Probabilità Smistamento (Telemetria)	60.00 %	59.97 %	- 0.03 %

Parametro Stocastico	Valore Atteso (Teorico)	Valore Rilevato (Empirico)	Scostamento / Errore Relativo
Media Servizio Navigazione (RVG)	30.00 ms	30.03 ms	0.103 %
Tempo di Interarrivo Medio	2.0000 ms	1.9926 ms	- 0.0074 ms
Tempo di Servizio Medio Totale	15.5000 ms	15.5188 ms	+ 0.0188 ms

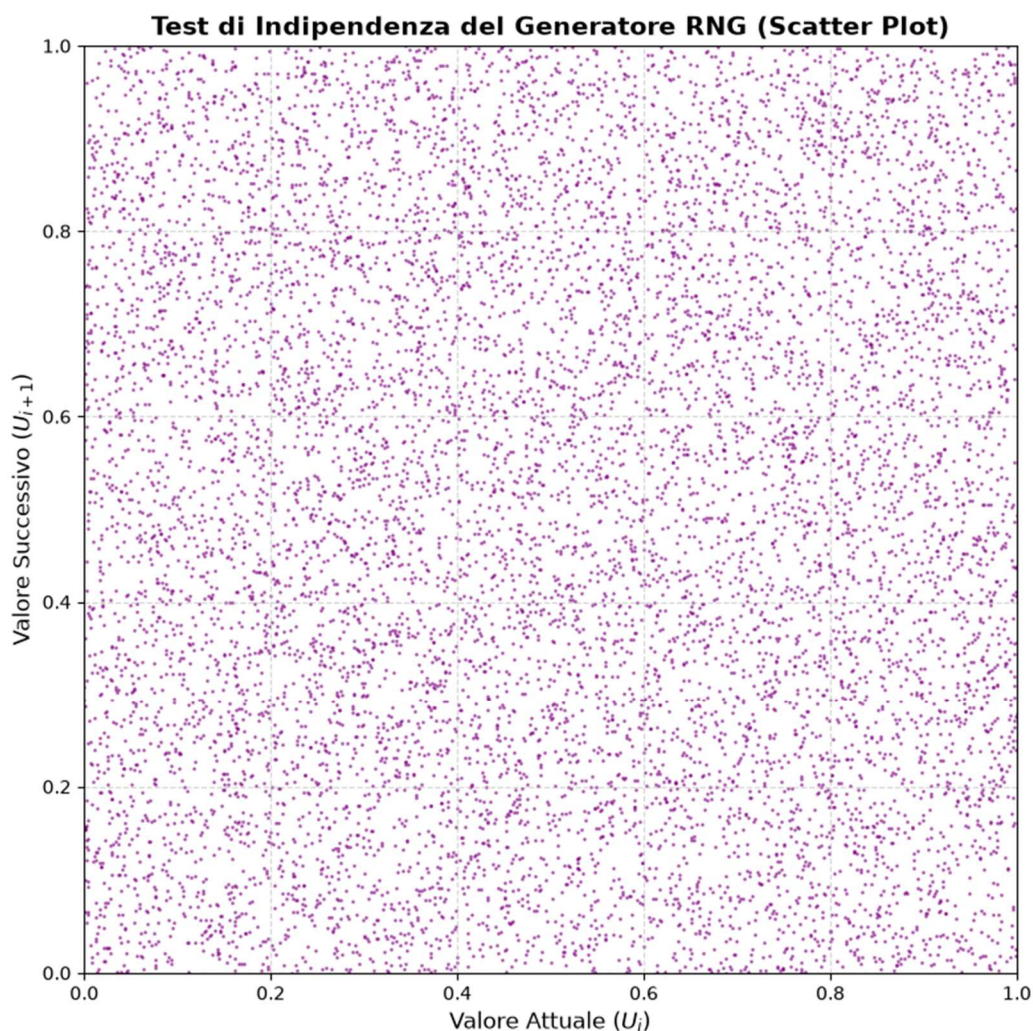


Figura 3: Test di Indipendenza del Generatore RNG (Scatter Plot). La mappatura delle estrazioni sequenziali conferma visivamente l'uniformità e l'indipendenza del generatore pseudocasuale implementato nel motore di simulazione, garantendo l'assenza di correlazioni spurie.

4.2 Validazione del Modello

In questa fase, l'obiettivo è dimostrare che l'impianto stocastico è esente da difetti e che il sistema, nel suo complesso, reagisce in modo logico e coerente con la teoria delle code. Mentre la verifica ha accertato l'assenza di bug informatici, la validazione ha lo scopo di certificare che il modello matematico rappresenti fedelmente l'architettura Edge reale, testandone il rigore statistico, la risposta dinamica sotto stress e l'aderenza analitica ai teoremi fondamentali.

4.2.1 Analisi del Carico e Stabilità

Il primo passo per validare il modello consiste nel confrontare i risultati empirici con i principi della Teoria delle Code. Nel simulatore, l'inter-arrivo medio globale è stato impostato a 2.0 ms. Questo parametro genera un tasso di arrivo globale teorico pari a:

$$\lambda_{tot} = \frac{1}{2.0} = 0.5 \text{ richieste/ms}$$

Conoscendo la ripartizione probabilistica delle classi e i rispettivi tempi di servizio attesi ($E[S_i]$), è possibile calcolare il traffico teorico offerto da ciascuna singola classe applicando la formula $A_i = \lambda_i \cdot E[S_i]$:

- Emergenza (10%):

$$\lambda_E = 0.10 \cdot 0.5 = 0.05 \text{ req/ms}$$

$$A_E = 0.05 \cdot 5.0 \text{ ms} = 0.25 \text{ Erlang}$$

- Navigazione (30%):

$$\lambda_N = 0.30 \cdot 0.5 = 0.15 \text{ req/ms}$$

$$A_N = 0.15 \cdot 30.0 \text{ ms} = 4.50 \text{ Erlang}$$

- Telemetria (60%):

$$\lambda_T = 0.60 \cdot 0.5 = 0.30 \text{ req/ms}$$

$$A_T = 0.30 \cdot 10.0 \text{ ms} = 3.00 \text{ Erlang}$$

Il Carico Totale Offerto al nodo Edge si ottiene sommando i tre contributi:

$$A_{tot} = 0.25 + 4.50 + 3.00 = 7.75 \text{ Erlang}$$

Conclusione: La Teoria delle Code impone una condizione fisica e matematica fondamentale: per mantenere un sistema stabile ed evitare la congestione infinita, il numero di server c deve essere strettamente maggiore del carico totale offerto ($c > A_{tot}$). Nello scenario di base, il simulatore è stato configurato con un limite capacitivo di $c = 4$ server.

Essendo il valore 4 strettamente minore dei 7.75 Erlang fisicamente richiesti dal traffico in ingresso, la matematica decreta inevitabilmente che il sistema deve collassare. Il simulatore ha rispecchiato questa legge, producendo un output in cui la coda a priorità più bassa (la Telemetria) diverge all'infinito accumulando oltre 13.000 job in attesa (come mostrato in *Tabella 5* e *Figura 4*). L'allineamento tra il disavanzo capacitivo teorico calcolato e l'esplosione della metrica considerata convalida l'accuratezza analitica del modello.

Tabella 5: Riscontro Empirico dell'Instabilità Analitica ($c = 4$, STOP=100.000)

Metrica di Sistema	Valore Medio Coda	Comportamento Rilevato
Coda E (Emergenza)	0.32	Stabile (Tutelata dalla priorità assoluta)
Coda N (Navigazione)	2.30	Stabile (Tutelata dall'offloading al 24.02%)
Coda T (Telemetria)	13090.79	Divergente (Saturazione delle risorse)

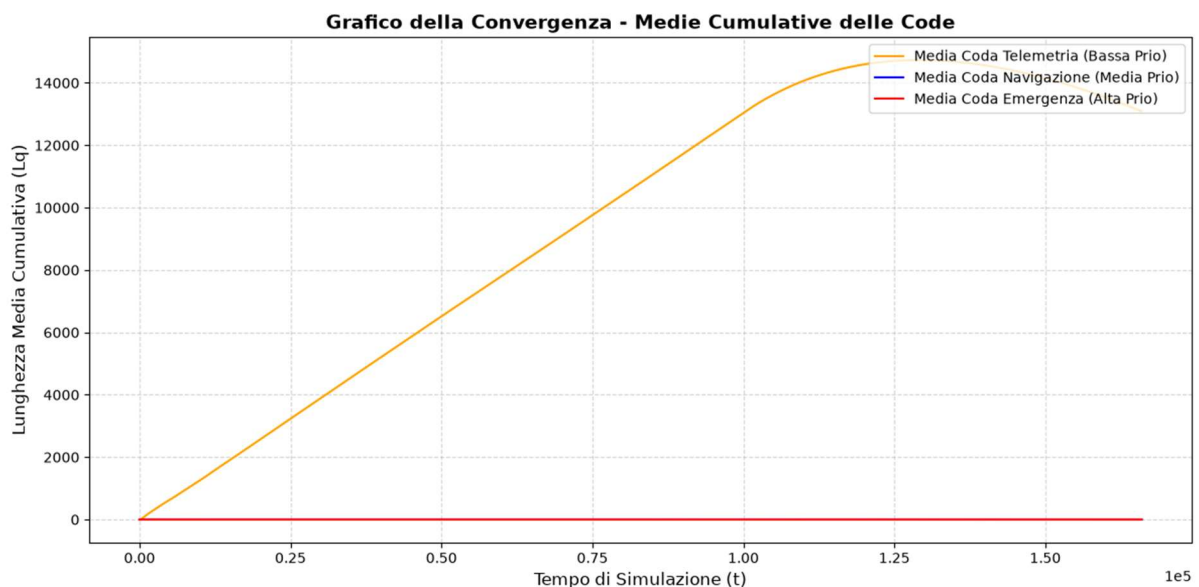


Figura 4: Analisi del transitorio in condizioni di instabilità ($c = 4$). Il grafico delle medie cumulative conferma l'assenza di uno steady-state per la coda di Telemetria (linea arancione). Come dimostrato dal calcolo del carico offerto ($c < A_{tot}$), la saturazione fisica dei server genera una divergenza illimitata, validando il comportamento fisico del modello.

4.2.2 Analisi di Sensibilità (Controlli di Coerenza)

Per validare la robustezza logica del simulatore, è stata condotta un'Analisi di Sensibilità. Questa tecnica non ricerca valori assoluti, ma verifica che le metriche di sistema reagiscano in modo coerente e proporzionato alle variazioni dei parametri di input. Modificando una singola variabile alla volta rispetto alla configurazione di base (4 server, soglia a 50.0 ms, inter-arrivo a 2.0 s), sono stati eseguiti tre scenari di test:

- Test di Coerenza sulla Capacità (Aumento dei Server): Al solo scopo di validare la logica di smaltimento del codice, la capacità è stata provvisoriamente sovrastimata a 10 server, garantendo un rapido assorbimento del traffico (la ricerca del limite capacitivo ottimale sarà invece oggetto del dimensionamento nell'Obiettivo 1). Come atteso dalla teoria, l'abbondanza di risorse ha stabilizzato il sistema: la coda di Telemetria è crollata a un valore medio di 3.97. Inoltre, l'algoritmo di deviazione non è mai scattato (tasso di offloading allo 0.00% e zero job deviati), confermando che il codice alloca le risorse parallele in modo corretto.
- Test sull'Offloading Dinamico (Abbassamento della Tolleranza): Ripristinata la capacità di base a 4 server, è stata testata la reattività dell'algoritmo abbassando la soglia dell'Expected Workload a 15.0 ms. Rendendo il sistema altamente "intollerante" all'accumulo, il tasso di offloading è balzato al 31.20%, deviando verso il nodo adiacente ben 4675 richieste di Navigazione. Questo intervento tempestivo ha compresso la coda media di Navigazione a soli 0.87 job, dimostrando che la formula di stima funge correttamente da valvola di pressione dinamica.
- Test sul Carico di Traffico (Stress Test degli Arrivi): Sempre mantenendo la configurazione a 4 server, l'intensità del traffico è stata quadruplicata, generando un volume massiccio di 200164 arrivi totali. Sotto questo sovraccarico, il sistema ha reagito deviando 50067 job di Navigazione (raggiungendo un tasso di offloading estremo dell'83.05%). Mentre la coda di Telemetria è esplosa in modo incontrollato accumulando in media 60015.63 richieste in attesa, la coda delle Emergenze è rimasta stabile a un valore medio di appena 0.57. Questo comportamento certifica in modo inequivocabile che la disciplina di priorità protegge i servizi critici, indipendentemente dalla congestione delle classi inferiori.

I risultati di queste verifiche sono riassunti nella *Tabella 6*:

Tabella 6: Sintesi dei Controlli di Coerenza (STOP=100.000)

Scenario di Test	Modifica Parametrica (Rispetto alla base)	Reazione Rilevata dal Sistema	Esito Validazione
Capacità Aumentata	10 server (invece di 4)	Coda Telemetria stabilizzata (3.97). Offloading azzerato (0.00%).	Coerente (Risorse sufficienti, congestione risolta).
Tolleranza Ridotta	Soglia attesa a 15.0 ms	Offloading incrementato al 31.20%. Coda Navigazione compressa (0.87).	Coerente (Algoritmo reattivo alla saturazione prevista).
Stress Test Traffico	Arrivi quadruplicati (200164)	Emergenza stabile (0.57). Telemetria divergente (60015.63).	Coerente (Sistema di priorità intatto sotto sforzo).

4.2.3 Validazione Analitica Globale

Questa sezione rappresenta la chiusura del processo di validazione, in cui il calcolo empirico del simulatore viene incrociato con i fondamenti teorici per certificare la correttezza matematica del motore stocastico sull'intero sistema.

- Il Test sull'Utilizzo (ρ): Come primo step, è stato verificato il bilancio di carico del nodo con una configurazione con 9 server. L'utilizzazione calcolata empiricamente dal simulatore tramite le variabili continue è risultata pari a 0.8656. Questo valore si allinea quasi perfettamente all'utilizzo teorico atteso, calcolato analiticamente come rapporto tra il traffico offerto e la capacità di smaltimento, pari a 0.8611.
- La Legge di Little Globale: Al fine di validare l'esattezza macroscopica del sistema, è stata verificata l'identità fondamentale della Teoria delle Code ($L = \lambda W$ e $L_q = \lambda W_q$) sull'intero aggregato del nodo Edge. Per garantire la coerenza matematica, al posto del tasso di arrivo nominale, è stato impiegato il tasso di arrivo effettivo (λ_{eff}) per decurtare esplicitamente i job scartati dalla logica di offloading dinamico. I risultati (mostrati nella *Tabella 7*) confermano che il calcolo discreto (matematicamente derivato dalle attese) combacia con il calcolo continuo estratto dagli integrali delle "aree temporali" del motore Next-Event.
- La Conservazione del Tempo: Un ulteriore test di coerenza interna ha dimostrato che la somma dell'attesa media in coda (35.6307 ms) e del tempo medio di servizio (15.5303 ms) restituisce esattamente il tempo di permanenza totale nel nodo (51.1610 ms).

Conclusione: Il perfetto parallelismo tra le formule teoriche e i risultati dei test certifica che le variabili di accumulo (*Time-Integrated Variables*) sono implementate correttamente nel codice C. Questa prova algebrica dimostra l'assoluta stabilità del sistema e l'assenza di perdite di precisione decimale durante il calcolo degli integrali, concludendo positivamente l'intero processo di validazione del modello.

Tabella 7: Validazione Analitica Globale del Nodo Edge (c=9, STOP=1.000.000)

Dominio Spaziale	Metrica Calcolata	Metrica Misurata	Esito Test
Intero Nodo (L)	25.6632	25.6632	Validato
Coda Aggregata (L_q)	17.8729	17.8729	Validato

5. Progettazione degli Esperimenti e Risultati

Dopo aver completato le fasi di verifica e validazione, il modello computazionale è stato utilizzato a tutti gli effetti come un banco di prova sperimentale. Questo capitolo illustra l'architettura degli esperimenti condotti per valutare le prestazioni del nodo Edge e risponde ai due quesiti fondamentali del progetto. Nelle sezioni seguenti verrà dapprima definita la metodologia statistica adottata per l'estrazione e l'analisi dei dati, necessaria per garantire l'affidabilità delle misurazioni. Successivamente, verranno presentati e discussi i risultati ottenuti nei due scenari di riferimento: il dimensionamento dell'infrastruttura in condizioni di traffico nominale (Obiettivo 1) e l'analisi transitoria di un evento di congestione estrema, valutando l'efficacia delle logiche di instradamento dinamico per la resilienza del sistema (Obiettivo 2).

5.1 Metodologia di Inferenza Statistica

Data la natura stocastica del modello, una singola esecuzione del simulatore non fornisce alcuna garanzia statistica sul comportamento medio del sistema reale. Per ovviare a questo limite e ottenere stime affidabili, la raccolta dei dati è stata automatizzata tramite uno script Python che orchestra le esecuzioni del codice C, adottando due distinte metodologie in base all'orizzonte temporale dello studio.

Metodo Batch Means (Analisi a Regime)

Per lo studio del dimensionamento del sistema in condizioni di normale operatività stazionaria (*Steady-State*), la raccolta dati si basa sul metodo delle Medie a Blocchi. Poiché nelle simulazioni a orizzonte infinito lo stato iniziale (sistema vuoto) introduce un bias artificiale, ripetere molteplici repliche comporterebbe lo spreco computazionale di dover scartare iterativamente la fase di transitorio iniziale. In questo scenario, l'eseguibile è configurato per compiere una singola simulazione estremamente prolungata nel tempo. Dopo aver scartato i dati della fase di riscaldamento (*Warm-up*), l'orizzonte temporale utile viene suddiviso in k blocchi adiacenti di ampiezza prefissata. L'aggregazione delle medie di ciascun blocco fornisce un campione di osservazioni che, grazie all'adeguata ampiezza scelta, risultano empiricamente non correlate (come confermato dal calcolo di un'autocorrelazione lag-1 pari a -0.0233).

Metodo delle Repliche Indipendenti (Analisi Transitoria)

Per l'analisi di eventi di crisi, caratterizzati da un orizzonte temporale finito (come l'impatto transitorio dell'incidente stradale), la simulazione adotta il metodo delle Repliche Indipendenti. Lo script Python è stato programmato per lanciare l'eseguibile C in modo automatizzato per n di volte, un numero dimensionato e validato preventivamente tramite la Procedura a Due Stadi. A ogni iterazione, l'ambiente Python passa da riga di comando un seed incrementale e differente per l'inizializzazione del generatore casuale. Questo approccio garantisce che gli esperimenti siano indipendenti tra loro, permettendo di isolare un campione di osservazioni non correlate su cui effettuare analisi statistica.

Sincronizzazione tramite Common Random Numbers (CRN)

Per confrontare il comportamento delle diverse architetture, è stata implementata la tecnica di riduzione della varianza dei *Common Random Numbers*. Nella fase di confronto tra scenari (ad esempio nell'Obiettivo 2, valutando il sistema di *baseline* contro quello ottimizzato dall'offloading), lo script inietta le stesse identiche sequenze di seed in entrambe le configurazioni. Sfruttando la rigida separazione dei flussi stocastici implementata nel codice C tramite la funzione `SelectStream`, la tecnica CRN impone che le due varianti architetturali subiscano lo stesso identico pattern di arrivi e le stesse fluttuazioni nei tempi di servizio. In questo modo qualsiasi variazione prestazionale diventa esclusivamente imputabile all'efficacia della politica di routing dinamico.

Calcolo degli Intervalli di Confidenza

Nello studio dei sistemi stocastici, affidare l'analisi a una singola stima puntuale (come la semplice media campionaria) risulta metodologicamente incompleto, poiché nasconde l'incertezza intrinseca dei dati. Per questo motivo, l'intera valutazione prestazionale del nodo Edge è ancorata al calcolo degli Intervalli di Confidenza. Lo script di automazione elabora gli estremi dell'intervallo garantendo un livello di confidenza del 95%. Lavorando su un campione limitato di n osservazioni indipendenti (rappresentate dalle singole iterazioni nel metodo delle Repliche e dai singoli blocchi temporali nel metodo Batch Means) e avendo una varianza della popolazione incognita, il calcolo impiega i quantili della distribuzione t di Student in luogo della Normale standard. La procedura garantisce che, ripetuta su infiniti campioni, il 95% degli intervalli così costruiti contenga il vero valore medio delle prestazioni, fornendo così un'indicazione quantificata dell'incertezza utile per il successivo dimensionamento dell'infrastruttura

5.2 Analisi Obiettivo 1: Dimensionamento Edge

L'obiettivo primario di questa prima fase di sperimentazione è determinare la configurazione hardware ottimale del nodo Edge, individuando il numero minimo di server necessari per gestire il traffico nominale. Il dimensionamento non ricerca la semplice stabilità del sistema, ma è vincolato dal rispetto dei criteri di sicurezza definiti nel Service Level Agreement (SLA): affinché il nodo sia considerato validato, la probabilità che la latenza di un pacchetto di Emergenza superi la soglia critica di 15 ms deve mantenersi al di sotto dell'1%.

5.2.1 Identificazione del Transitorio

Prima di procedere all'estrazione dei dati per il dimensionamento, è necessario garantire la neutralità statistica delle misurazioni, affrontando il problema del bias dello stato iniziale.

- Il Problema Teorico: Il motore di simulazione avvia il sistema da una condizione "vuota e inattiva", ovvero con tutte le code azzerate e i server completamente liberi. Di conseguenza, i primi pacchetti in ingresso trovano un nodo irrealisticamente scarico e vengono elaborati istantaneamente, sperimentando latenze quasi nulle. Includere questi dati eccessivamente ottimistici nel calcolo finale comporterebbe una distorsione (*bias*) della stima, falsando la valutazione delle prestazioni del sistema a regime (*steady-state*).
- La Soluzione Grafica: Per individuare l'esaurimento di questa anomalia iniziale, è stato adottato il metodo dell'ispezione visiva sulle medie cumulative. Come mostrato nel Grafico della Convergenza (*Figura 5*), è stato tracciato l'andamento nel tempo della lunghezza delle code. Questa analisi è stata condotta campionando i dati dalla configurazione a 9 server, ovvero la prima architettura hardware testata in grado di garantire la stabilità matematica del sistema. Essendo la configurazione più "sotto sforzo" tra quelle stabili, garantisce una stima conservativa e cautelativa del transitorio. Osservando la metrica più sensibile (la coda di Telemetria, in arancione), si nota chiaramente una prima fase di forte oscillazione che si smorza progressivamente. Il punto di flesso, in cui il sistema abbandona il transitorio per assestarsi attorno a valori di regime, è stato individuato visivamente tracciando un limite verticale in corrispondenza del tempo $t = 500000$.
- L'Implementazione (Troncamento dei Dati): Questa osservazione fenomenologica è stata tradotta in una logica di controllo all'interno del codice sorgente in C. L'istante di stabilizzazione è stato codificato tramite la costante WARMUP, impostata al valore di

500.000 ms. Dal punto di vista algoritmico, non appena l'orologio della simulazione incrocia questa soglia temporale, interviene un blocco condizionale governato dal flag `warmup_done`. Questo meccanismo provvede ad azzerare i contatori di latenza accumulati fino a quel momento (le variabili `completed_E` e `violations_E`). Grazie a questa procedura di troncamento, il programma inizia a collezionare statistiche "pulite" esclusivamente quando il nodo Edge si trova a pieno regime operativo, garantendo stime affidabili per la valutazione dello SLA.

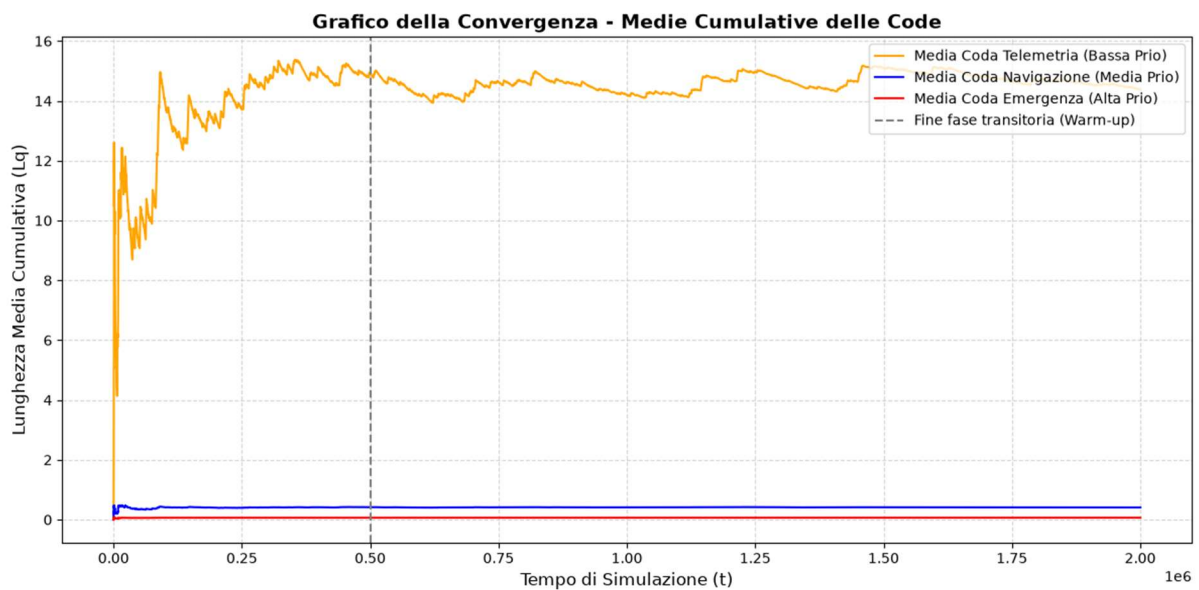


Figura 5: **Grafico della Convergenza e determinazione del Warm-up ($c=9$)**. L'andamento delle medie cumulative evidenzia l'esaurimento del bias dello stato iniziale (*Empty and Idle*). La linea tratteggiata verticale a $t = 500.000$ marca il confine tra la fase transitoria e il raggiungimento del regime stazionario (*steady-state*).

5.2.2 Risultati e determinazione del numero minimo di server

Progettazione dell'Esperimento

Per determinare il corretto dimensionamento del sistema, la sperimentazione è stata strutturata definendo le variabili in gioco e le condizioni al contorno:

- Fattore di Input: La variabile indipendente manipolata è la capacità di calcolo del nodo Edge, ovvero il numero di server c , testato per step incrementali partendo da 8 unità.
- Condizioni Fisse: Per non invalidare l'esperimento, il tasso di arrivo globale λ è stato mantenuto al valore di traffico nominale, la logica di offloading è rimasta invariata (soglia a 50 ms) e il tempo di simulazione è stato impostato a $STOP = 25.500.000$ ms per garantire l'estrazione di dati abbondanti a regime.
- Variabile di Risposta: La metrica valutata è la probabilità di violazione dello SLA, espressa matematicamente come $P(R_E > 15\text{ ms})$, dove R_E è il tempo totale di permanenza di un segnale di Emergenza.
- Solidità Statistica: Data la severità del vincolo (tolleranza massima dell'1%), per ogni configurazione hardware sono state estratte 50 osservazioni tramite il metodo Batch Means. Su tali dati è stato calcolato l'Intervallo di Confidenza al 95%.

Presentazione dei Dati

I risultati delle simulazioni, calcolati aggregando i 50 blocchi indipendenti per ogni scenario analizzato, sono riassunti nella *Tabella 8* e visualizzati graficamente nella *Figura 6*.

Tabella 8: Riepilogo delle probabilità medie di violazione del vincolo di latenza

Server (c)	Batch	Probabilità Media Violazione	Intervallo di Confidenza (95%)	Esito SLA (< 1%)
8	50	2.6662%	[2.6134%, 2.7190%]	FALLITO
9	50	1.0706%	[1.0416%, 1.0997%]	FALLITO
10	50	0.4183%	[0.4015%, 0.4352%]	SUCCESSO
11	50	0.1493%	[0.1396%, 0.1591%]	SUCCESSO

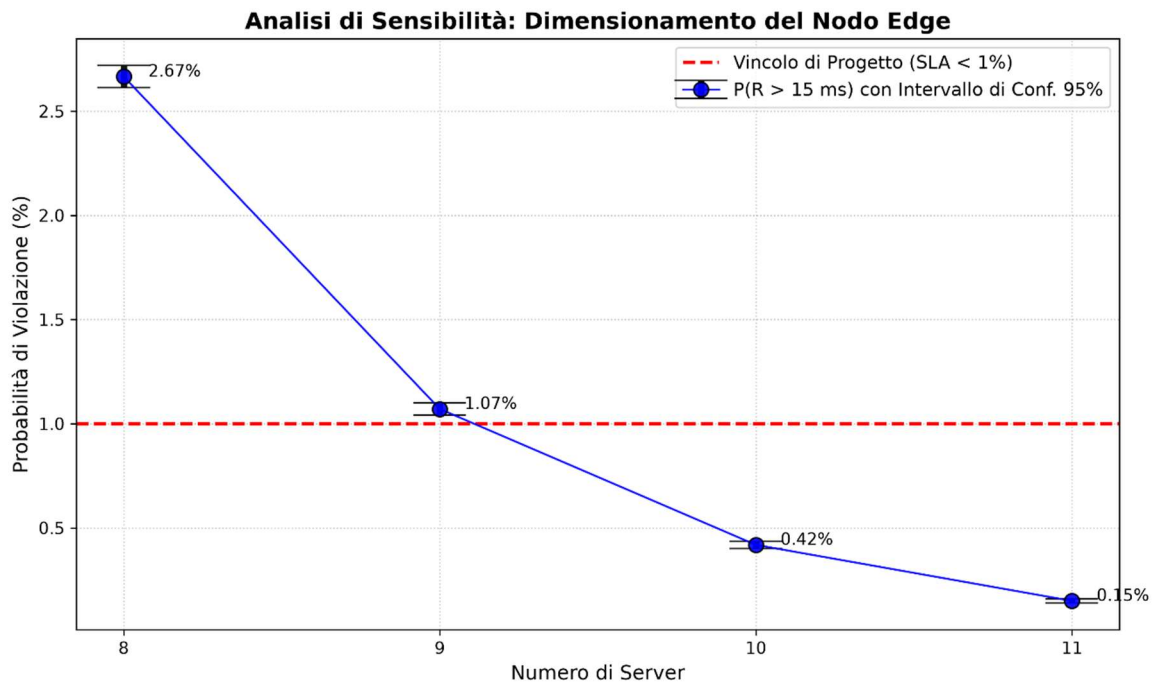


Figura 6: *Analisi di sensibilità per il dimensionamento del nodo Edge. L'andamento della probabilità di violazione (curva blu) evidenzia il superamento del limite dell'1% (linea tratteggiata rossa) per le configurazioni a 8 e 9 server, e il raggiungimento del target a partire da 10 server.*

L'Analisi e la Scelta

I dati estratti dimostrano la netta distinzione tra la semplice "stabilità" di un sistema a code e il reale soddisfacimento dei vincoli di Qualità del Servizio (QoS).

- Il limite della stabilità (9 Server): Sebbene la configurazione a 9 server sia teoricamente sufficiente a garantire la stabilità matematica (evitando la divergenza infinita delle code), le prestazioni non sono adeguate. A causa dell'alta varianza insita nel traffico di Navigazione, il sistema subisce micro-congestioni che portano l'1.07% dei pacchetti di Emergenza a violare il tempo limite.
- La configurazione ottima (10 Server): Aggiungendo un solo server aggiuntivo al cluster, la probabilità media di violazione subisce un crollo drastico, attestandosi allo 0.4183%. Essendo il limite superiore dell'intervallo pari a 0.4352%, il vincolo dell'1% risulta ampiamente soddisfatto. Questa architettura rappresenta il dimensionamento perfetto per la rete nominale analizzata.
- Prevenzione dell'Over-provisioning (11 Server): L'ultimo scenario dimostra che scalare ulteriormente l'infrastruttura a 11 server abbate le violazioni allo 0.1493%. Tuttavia, essendo il vincolo contrattuale dello SLA già garantito dalla configurazione precedente, tale aggiunta si tradurrebbe in un caso di *over-provisioning*.

5.3 Analisi Obiettivo 2: Congestione Dinamica

In questa sezione viene affrontato il secondo obiettivo del progetto, spostando l'attenzione dall'analisi a regime stazionario allo studio dell'evoluzione transitoria generata da un evento anomalo. Fissata l'architettura ottima a 10 server, come dedotto nel precedente dimensionamento, il nodo Edge viene sottoposto a uno *stress test* che simula le conseguenze di un incidente stradale. Tale evento innesca un improvviso *burst* che quadruplica il tasso di arrivo globale e altera drasticamente la composizione del traffico, saturando il sistema con un'ondata di Segnali di Emergenza e richieste di ricalcolo del percorso (Navigazione).

L'obiettivo dell'esperimento è misurare la resilienza dell'infrastruttura e quantificare l'efficacia della logica di *Dynamic Offloading* basata sul calcolo dell'attesa stimata (*Expected Workload*). Per dimostrare il valore di questa politica di controllo, l'analisi è stata strutturata in due scenari comparativi: un primo scenario di *Baseline*, utile a isolare il collasso fisiologico del nodo in assenza di difese, e un secondo scenario *Ottimizzato*, in cui l'algoritmo di deviazione del carico viene attivato per preservare i vincoli di sicurezza e i tempi di recupero del sistema.

5.3.1 Scenario di Baseline e Starvation

Per comprendere appieno il valore della politica di mitigazione, il sistema è stato inizialmente valutato in uno scenario di *Baseline*, in cui l'algoritmo di deviazione del traffico (*Offloading*) è stato artificialmente disattivato impostando una soglia limite irraggiungibile. In questa configurazione, il nodo Edge da 10 server è costretto ad assorbire l'intera ondata anomala scaturita dall'incidente, affidandosi esclusivamente alla disciplina di priorità.

I risultati di questa simulazione mettono in luce tre criticità architetturali:

1. La penalità della Non-Preemption sui servizi salvavita (SLA)

Nonostante la rigida priorità assoluta garantita ai Segnali di Emergenza, il vincolo di sicurezza ($SLA \leq 15$ ms) subisce un grave cedimento, registrando una probabilità media di violazione del 8.1906% (con un picco massimo di latenza pari a 94.09 ms). Questo decadimento prestazionale dimostra la cosiddetta "penalità della Non-Preemption". Essendo i 10 server non interrompibili, essi vengono rapidamente saturati dall'enorme mole di richieste di Navigazione in ingresso, una parte delle quali richiede pesanti elaborazioni fino a 110 ms. Di conseguenza, un pacchetto di Emergenza, pur scavalcando l'intera coda, è costretto ad attendere la naturale

liberazione di un server occupato da processi lenti, accumulando ritardi pesanti per un servizio critico.

2. Il collasso a cascata e il fenomeno della Starvation

L'assenza dell'algoritmo dell'offloading si ripercuote sui buffer di memoria delle code secondarie, rendendo visibile il fenomeno della Starvation.

- I pacchetti di Navigazione, non venendo intercettati e scartati preventivamente, si riversano nel nodo Edge generando una coda che raggiunge un picco medio di 392.007 unità. In un'infrastruttura reale, tale accumulo porterebbe all'esaurimento istantaneo della memoria RAM (*Buffer Overflow*) e al crash del server.
- La situazione precipita ulteriormente per la classe a priorità più bassa, la Telemetria. Dovendo attendere lo smaltimento sia del traffico di Emergenza che della coda di Navigazione, la Telemetria subisce un blocco totale dell'elaborazione, portando il buffer a esplodere fino a un picco di 821.604 pacchetti in attesa.

3. L'illusione del Recovery Time

Un'analisi superficiale del solo traffico critico indicherebbe un tempo di recupero (*Recovery Time*) eccellente per la coda di Emergenza, pari a soli 3.61 ms dalla fine dell'incidente. Tuttavia, questo dato è fuorviante. Tracciando il tempo necessario per svuotare l'intero sistema (compresi gli enormi arretrati di Navigazione e Telemetria), la simulazione rivela che il nodo Edge impiega in media 5890433.66 ms (oltre un'ora e mezza di tempo simulato) per ritornare alla normalità.

Come confermato dal grafico in *Figura 7*, disattivare le logiche di controllo dinamico condanna l'infrastruttura Edge a una saturazione prolungata, paralizzando la capacità di calcolo del nodo ben oltre la durata effettiva dell'incidente stradale.

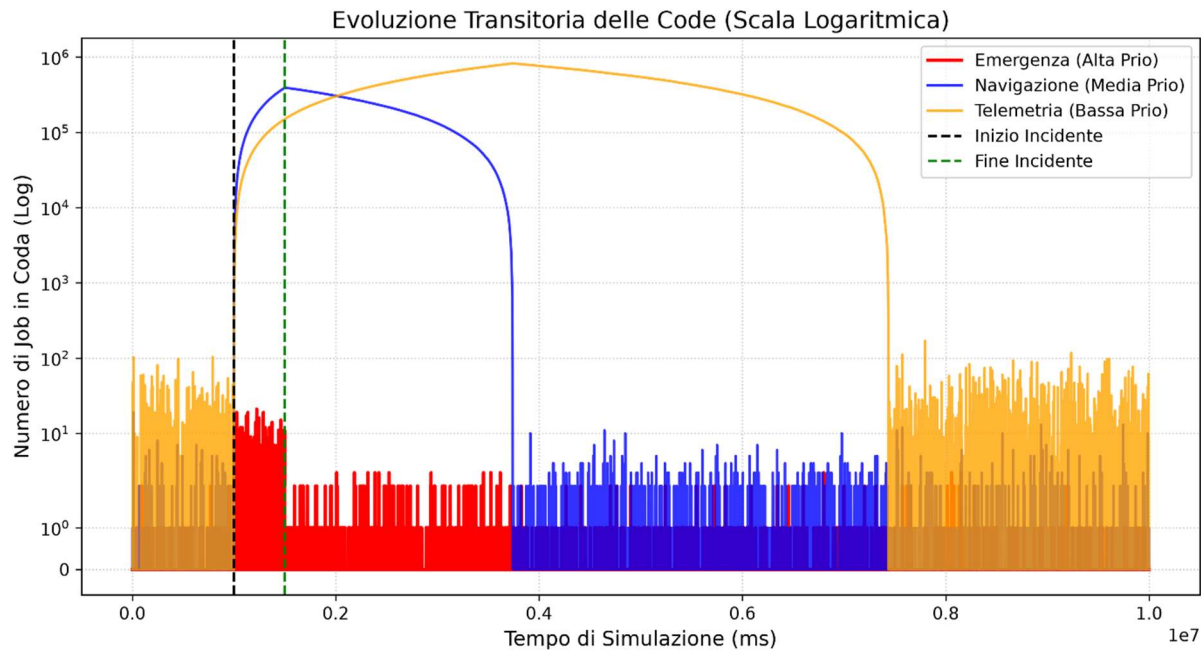


Figura 7: **Evoluzione transitoria delle code nello scenario di Baseline (Scala Logaritmica, $c=10$).** In assenza di offloading, la finestra dell'incidente (delimitata dalle linee tratteggiate) innesca un collasso globale. La coda di Navigazione (blu) e quella di Telemetria (arancione) divergono in modo incontrollato. È evidente come lo smaltimento del traffico in background richieda milioni di millisecondi ben oltre il termine dell'emergenza fisica.

5.3.2 Scenario Ottimizzato e Analisi del Transitorio

Progettazione dell'Esperimento

Per garantire la validità scientifica del confronto tra i due scenari (con e senza offloading), la campagna di simulazione transitoria è stata progettata secondo i seguenti criteri:

- Fattori di Input e Risposte: Mantenendo fissa l'infrastruttura a 10 server, la variabile indipendente manipolata è lo scenario di traffico. All'istante $t = 1.000.000$ (inizio incidente), il tasso di arrivo viene quadruplicato e la composizione alterata ($p_E = 0.35$, $p_N = 0.50$ e $p_T = 0.15$). Lo stato di crisi rientra all'istante $t = 1.500.000$. Le variabili di risposta misurate sono il picco massimo di latenza per le Emergenze, il tasso di deviazione e il *Recovery Time* (il tempo di smaltimento delle code post-incidente).
- Gestione del Bias di Inizializzazione: Per evitare di valutare un evento anomalo su un sistema vuoto e irrealistico (partenza a $t = 0$), l'incidente non viene innescato immediatamente. Il simulatore elabora il traffico nominale per 1.000.000 di millisecondi, permettendo al sistema di raggiungere il regime stazionario, e solo in quel momento viene iniettato il picco di traffico.
- Finestra di Valutazione e Prevenzione della Diluizione: Per evitare l'alterazione statistica nota come *Effetto Diluizione*, l'orizzonte di simulazione è stato disaccoppiato dalla finestra di campionamento. Sebbene l'esecuzione prosegua fino a $t = 10.000.000$ ms per tracciare il *Recovery Time* del sistema di base, il calcolo della probabilità di violazione SLA e del tasso di offload viene interrotto a $t = 3.000.000$ ms. Ciò permette di isolare le metriche esclusivamente durante la fase di acuta congestione e di successivo smaltimento.
- Inferenza e Confronto Equo (CRN): Trattandosi di un evento transitorio isolato, l'inferenza è stata condotta tramite 50 repliche indipendenti. Fondamentale è stata l'applicazione della tecnica dei Common Random Numbers. Grazie all'uso di stream stocastici separati (SelectStream), si è garantito che il nodo affrontasse l'esatta stessa sequenza di pacchetti in ingresso in entrambi gli scenari, assicurando che le differenze prestazionali misurate fossero imputabili unicamente all'efficacia della logica di controllo.

Presentazione dei Dati

Per evidenziare i benefici introdotti dalla politica di controllo, i risultati estratti dalla simulazione dello scenario *Ottimizzato* sono stati messi a confronto diretto con la *Baseline* nella seguente tabella riassuntiva (*Tabella 9*).

Tabella 9: Confronto Prestazionale durante l'Incidente (c=10)

Metrica Analizzata	Scenario Baseline (Senza Offload)	Scenario Ottimizzato (Con Offload)	Beneficio
Pacchetti di Navigazione deviati	0	391.995	Salvataggio Memoria
Prob. Violazione Emergenze	8.1906% $\pm$ 0.064%	7.0628% $\pm$ 0.062%	-1.1278%
Picco Latenza Emergenze	94.09 ms $\pm$ 4.38 ms	69.02 ms $\pm$ 4.30 ms	-25.07 ms
Picco Coda Navigazione	392.007 pacchetti	17 pacchetti	Evitato Overflow
Picco Coda Telemetria	821.604 pacchetti	150.157 pacchetti	-82% Starvation
Recovery Time Globale	~5.89 milioni di ms	~668 mila ms	-89% Tempo

Analisi del Transitorio e Risultati

L'attivazione dell'algoritmo di deviazione trasforma radicalmente la reazione dell'infrastruttura Edge allo "shock" causato dalla congestione, come è possibile evincere dall'analisi dei dati e dal grafico transitorio (*Figura 8*).

- Il Taglio della Navigazione: Nello scenario ottimizzato, il picco massimo della coda di Navigazione si arresta esattamente a 17 pacchetti. Questo taglio orizzontale dimostra l'efficacia della condizione algoritmica: non appena la stima dell'attesa supera la soglia, il sistema chiude gli ingressi deviando circa 392.000 richieste. Questa azione funge da salvavita per il nodo, impedendo il *Buffer Overflow* osservato nella Baseline.

- Riduzione della Latenza: Sebbene i pacchetti di Emergenza abbiano sempre la priorità massima, l'offloading migliora sensibilmente le loro prestazioni, abbattendo il picco di latenza da circa 94 ms a 69 ms e riducendo le violazioni. Deviando il massiccio volume di Navigazione, l'algoritmo riduce la probabilità che i 10 server rimangano ostaggio di elaborazioni lunghe (110 ms), garantendo che le Emergenze trovino processori liberi molto più rapidamente.
- Mitigazione della Starvation e del Recovery Time: Il beneficio maggiore si riflette sul collasso a cascata. Pur subendo comunque un'attesa fisiologica, la coda della Telemetria riduce il suo picco da oltre 821.000 a 150.157 unità. Soprattutto, aver evitato l'accumulo in memoria permette al sistema di smaltire l'intero arretrato e ritornare allo stato stazionario in circa 668.000 ms, un tempo dieci volte inferiore rispetto quasi 6 milioni di millisecondi necessari senza offloading.

In conclusione, l'esperimento certifica che l'implementazione del *Dynamic Offloading* basato sull'*Expected Workload* è essenziale non solo per ottimizzare i tempi del traffico critico, ma per garantire la sopravvivenza e la resilienza del nodo Edge durante eventi imprevedibili.

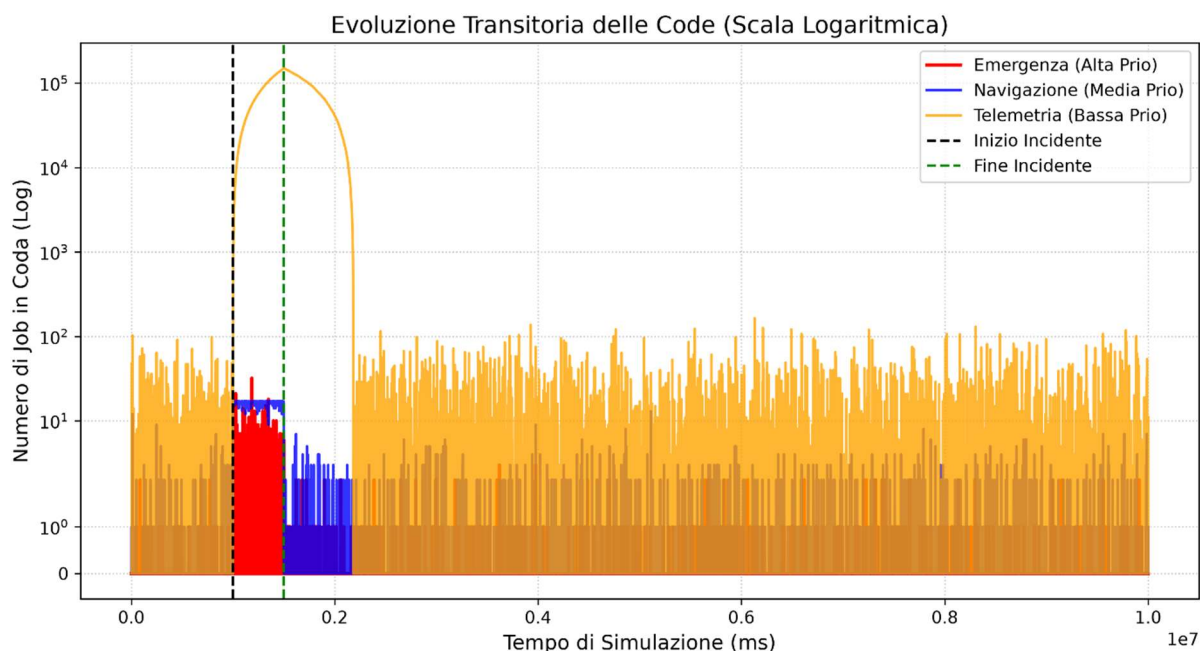


Figura 8: *Evoluzione transitoria delle code nello scenario Ottimizzato (Scala Logaritmica, $c=10$)*. È chiaramente visibile il taglio orizzontale netto imposto alla coda di Navigazione (linea blu), che viene bloccata al raggiungimento del limite di soglia. Di conseguenza, la Telemetria (linea arancione) subisce una starvation nettamente inferiore e il sistema recupera il regime stazionario molto più rapidamente dopo la fine dell'incidente.

6. Evoluzione del Modello

L'analisi dello scenario di congestione dinamica ha messo in luce i limiti fisici e architetturali del sistema. Sebbene la deviazione del traffico abbia mitigato il collasso totale del nodo Edge, il sistema ha mostrato una criticità inaccettabile per un ambiente di *Smart City*: le violazioni della latenza per i pacchetti di Emergenza si sono attestate su valori critici (superiori al 7% con 10 server e costantemente sopra il 3% anche scalando a 12 server), fallendo il vincolo di sicurezza. Per rendere l'infrastruttura realmente resiliente, il sistema è stato sottoposto a un processo di riprogettazione logica. Di seguito vengono ripercorsi i passi metodologici che hanno guidato questa evoluzione.

Formulazione del Problema (Definizione degli Obiettivi)

L'obiettivo di questa fase evolutiva è abbattere la probabilità di violazione della latenza per i pacchetti di Emergenza. Il sistema deve essere in grado di rientrare stabilmente al di sotto della soglia contrattuale dell'1% di violazioni SLA, assicurando un instradamento istantaneo e la massima tutela per i segnali di Emergenza, anche durante i picchi massimi di congestione.

Modello Concettuale

Il modello concettuale di base è stato evoluto introducendo una tecnica nota in Teoria delle Code come *Adaptive Trunk Reservation* (Figura 9). L'algoritmo non altera la natura delle entità o il numero di risorse disponibili, ma interviene direttamente sulla disciplina di smistamento del server tramite un approccio *State-Dependent Scheduling*.

- Il Principio Logico: Fissata la capacità computazionale totale del nodo (c server), viene introdotta una soglia limite K (con $K < c$) applicata esclusivamente al traffico di classe 2 (Navigazione) e 3 (Telemetria). Se il numero di server attualmente occupati dall'elaborazione di queste due classi raggiunge il limite K , eventuali nuovi pacchetti di background vengono forzatamente messi in attesa, anche in presenza di server liberi. In questo modo, l'algoritmo crea un *gap* di $c - K$ server riservati per l'elaborazione istantanea delle Emergenze.
- Attivazione Dinamica (Crisis Mode): Mantenere questa riserva costantemente attiva genererebbe un fenomeno noto come *Artificial Starvation* (sotto-utilizzazione artificiale) durante i periodi di traffico nominale, causando sprechi di CPU e incrementando inutilmente la latenza del traffico di background. Pertanto, l'algoritmo è stato dotato di una logica "adattiva": il blocco dei server entra in funzione

esclusivamente quando l'algoritmo rileva un'anomalia di rete, ovvero quando l'equazione del ritardo atteso ($E[W]$) supera una soglia di pre-allarme.

- Gestione del Transitorio di Rientro (Soft Recovery): Il disinnescamento del blocco presenta una sfida computazionale. Se la riserva venisse disattivata istantaneamente al cessare dell'incidente fisico, i pacchetti in attesa si riverserebbero immediatamente sui server liberi. Questo sequestro simultaneo della CPU causerebbe latenze critiche per le Emergenze che arrivano nei millisecondi successivi. Per prevenire questo disservizio auto-inflitto, il modello logico prevede una disattivazione ritardata (tramite doppia soglia) e un rilascio graduale (*Soft Recovery*): il limite K viene innalzato progressivamente man mano che la coda si svuota, garantendo che ci sia costantemente un nodo libero per tutelare i servizi salvavita durante la fase di assestamento post-incidente.
- Definizione e Dinamica delle Variabili di Stato: Per implementare l'Adaptive Trunk Reservation, il modello si arricchisce di tre nuove variabili di stato: il flag di attivazione della Crisis Mode, il limite hardware per il traffico non critico (K), e il contatore dei server fisicamente occupati da Navigazione e Telemetria. Queste variabili sono logicamente concatenate: l'attivazione dello stato di crisi dipende dalla lunghezza istantanea delle code prioritarie (valutata tramite l'equazione dell'attesa media stimata). Il superamento di una certa soglia di attesa innesca il controllo sul limite hardware K ; quest'ultimo agisce così da filtro protettivo, bloccando la crescita del contatore dei server disponibili per arrivi non critici e garantendo risorse libere per le emergenze.
- Gestione degli Eventi e Transizioni di Stato: L'evoluzione algoritmica non introduce nuove tipologie di eventi nella *Next-Event List*, mantenendo inalterata l'architettura basata su Arrivi e Partenze. Tuttavia, la logica di transizione di stato eseguita al verificarsi di tali eventi viene estesa:
 - *Evento di Arrivo*: Ogni nuovo ingresso innesca ora un calcolo attivo della metrica $E[W]$ per determinare dinamicamente lo stato di crisi e l'aggiornamento della soglia K . Solo successivamente il sistema valuta il partizionamento dei server per decidere l'ammissione o l'accodamento forzato del job.
 - *Evento di Partenza*: Diventa il motore del *Soft Recovery*. Al completamento di un servizio, prima di prelevare il job successivo, il sistema valuta il livello di

svuotamento delle code e aggiorna progressivamente la soglia K verso l'alto, regolando il rilascio graduale della capacità computazionale.

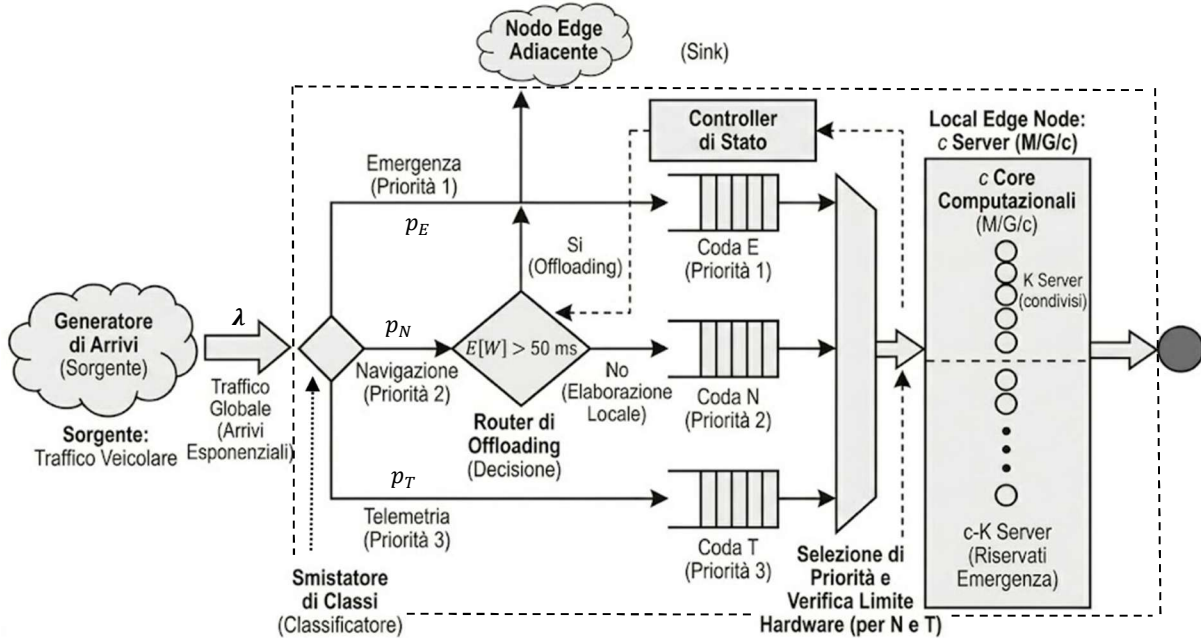


Figura 9: **Architettura logica con Adaptive Trunk Reservation.** Il sistema introduce un controllo per limitare l'accesso dei job di Navigazione e Telemetria a un massimo di K server durante i picchi di congestione, riservando le restanti risorse ($c - K$) esclusivamente al traffico salvavita.

Modello di Specifica

Le politiche concettuali di partizionamento sono state tradotte in equazioni matematiche e parametri deterministici, definendo le condizioni di attivazione della *Crisis Mode* e del *Soft Recovery*:

- Equazione di Rilevamento Doppia Soglia: Il passaggio di stato del nodo non è guidato dal tempo, ma da una funzione di costo basata sull'attesa stimata $E[W]$. Per evitare continue e instabili oscillazioni del sistema, la logica di specifica considera due soglie asimmetriche:
 - *Trigger di Attivazione:* Se $E[W] > 40 \text{ ms}$, il sistema entra in stato di Crisi.
 - *Trigger di Disattivazione:* Il sistema esce dall'allarme solo se $E[W] < 10 \text{ ms}$ e le code prioritarie sono strettamente vuote ($N_E = 0, N_N = 0$).

- Condizione di Ammissione (Trunk Reservation): Definito c il numero totale di server e c_{bg} il numero di server attualmente impegnati nell'elaborazione di traffico non critico (classi 2 e 3). Durante lo stato di Crisi, la variabile dinamica K viene impostata al limite restrittivo (es. $K = 8$). La logica di instradamento per un job di background in arrivo diventa:

$Se (c_{bg} < K) \rightarrow \text{Avvia Servizio}$

$Altrimenti \rightarrow \text{Accodamento Forzato}$

Questa disuguaglianza garantisce matematicamente un *gap* di $c - K$ server costantemente protetti per la classe di Emergenza.

- Logica di Rilascio (Soft Recovery): Per implementare il recupero graduale, la variabile K non riassume istantaneamente il valore c al termine dell'allarme. Durante la fase di svuotamento post-incidente, l'algoritmo di specifica prevede che K venga incrementato di una singola unità (es. da 8 a 9, poi a 10) solo al verificarsi congiunto di due condizioni: il completamento di un servizio e il mantenimento del vincolo di sicurezza sulle latenze stimate.

Modello Computazionale

Per questa evoluzione, l'algoritmo di *Adaptive Trunk Reservation* è stato integrato all'interno del simulatore sviluppato in linguaggio C, mantenendo l'approccio orientato agli eventi. L'architettura del software è stata espansa per supportare il nuovo controllo retroazionato senza alterare la struttura base della *Event List*, agendo unicamente sulle strutture dati di stato e sui blocchi condizionali di smistamento.

- Strutture Dati e Variabili di Stato: Per consentire al sistema di tracciare dinamicamente le partizioni hardware, sono state introdotte nuove variabili di stato globali nel *main* del programma:
 - Una variabile intera (*busy_servers_non_E*) dedicata esclusivamente al conteggio in tempo reale dei server occupati dalle classi di Navigazione e Telemetria.
 - Un parametro limite dinamico (*limit_non_E*), inizializzato al numero totale dei server fisici (c), che agisce da soglia mobile per l'accettazione dei job non critici.
 - Un *flag* booleano (*in_crisis_mode*) per tracciare lo stato del nodo (condizione di normalità o condizione di anomalia).

- Implementazione del Sensore di Crisi (Evento di Arrivo): Nel blocco di codice relativo al processamento degli arrivi, la logica computazionale è stata modificata per fungere da sensore di stato. Prima di ogni assegnazione, il programma calcola la stima del ritardo atteso. È stato inserito un blocco condizionale che valuta tale stima: se questa supera i 40 ms e il sistema non è già in stato di crisi, il *flag* viene sollevato e la variabile *limit_non_E* viene istantaneamente declassata. Questo passaggio implementa la "chiusura del cancello", riservando fisicamente i restanti server all'Emergenza. In fase di assegnazione ai server fisicamente liberi, è stato aggiunto un filtro discriminante: se il job corrente appartiene alla classe Navigazione o Telemetria, il sistema verifica che *busy_servers_non_E* sia strettamente minore di *limit_non_E*. In caso contrario, l'esecuzione del task viene interdetta e il pacchetto viene forzatamente dirottato nell'istruzione di accodamento, proteggendo il server *Idle*.
- Implementazione del Rilascio a Doppia Soglia (Evento di Partenza): Il blocco di codice deputato allo smaltimento dei job (else di gestione delle partenze) è stato modificato per implementare il *Soft Recovery*. Quando un job di classe 2 o 3 termina, il contatore *busy_servers_non_E* viene decrementato. A questo punto, il software esegue un controllo incrociato sui parametri di stabilità: se il *flag* di crisi è attivo, le code prioritarie (E ed N) sono vuote e l'attesa stimata è scesa sotto i 10 ms, il software incrementa la variabile *limit_non_E* di una singola unità, iterando questo processo a ogni partenza finché il limite non torna a coincidere con la capacità totale *c*.
- Modifica dello Schedulatore Prioritario: L'ultima fase del modello computazionale riguarda il prelievo dei job in coda (*Scheduling*). Le istruzioni if-else relative alla Navigazione e alla Telemetria sono state vincolate da una doppia condizione: il prelievo avviene solo se la rispettiva coda è maggiore di zero e simultaneamente la variabile *busy_servers_non_E* è inferiore al limite attuale *limit_non_E*. Se questa condizione non è verificata, la catena condizionale degrada verso l'istruzione finale (else), che decrementa i server totali occupati spegnendo il server e mettendolo a riposo in attesa di un eventuale arrivo di Emergenza, completando così la traduzione computazionale della *Trunk Reservation*.

Verifica

Per assicurarsi che il codice implementasse fedelmente la restrizione delle risorse e il controllo a doppia soglia, il simulatore è stato sottoposto a tecniche di test del software specifiche:

- Boundary Testing: Sono state effettuate esecuzioni forzando parametri estremi. Impostando il limite $K = 0$ durante la crisi, si è verificato che le code secondarie venissero totalmente bloccate e che i server agissero come un sistema puro a singola classe (solo Emergenza). Al contrario, forzando $K = c$, si è verificato che il sistema si comportasse esattamente come lo scenario non evoluto, dimostrando che il "cancello logico" risponde correttamente ai parametri immessi (risultati riportati in *Tabella 10*).
- Analisi della Traccia: Attraverso la generazione di log temporali testuali (campionamento degli stati), si è verificato l'aggiornamento del contatore dei server non critici. Questo ha garantito che l'istruzione condizionale di *scheduling* non permettesse mai alla classe di Navigazione o Telemetria di occupare $K+1$ server, confermando l'inviolabilità della partizione hardware (estratto dei risultati riportato in *Tabella 11*).

Tabella 10: Risultati del Boundary Testing sul parametro K (c=10)

Metrica Analizzata	Blocco Totale ($K = 0$)	Nessuna Restrizione ($K = 10$)
Prob. Violazione SLA (Emergenza)	$0.00\% \pm 0.00\%$	$7.06\% \pm 0.06\%$
Tasso di Deviazione (Navigazione)	$100.00\% \pm 0.00\%$	$54.04\% \pm 0.03\%$

Tabella 11: Estratto log per Analisi della Traccia (c=10, K=8)

Tempo (ms)	Coda Emergenza	Limite (K)	Server Non-Critici Occupati
1000000.0	0	10	10
1001000.0	2	8	8
1002000.0	1	8	4
1003000.0	1	8	5

Validazione

La validazione ha avuto lo scopo di dimostrare che le reazioni del modello fossero sensate rispetto all'introduzione della nuova politica:

- Ispezione Visiva del Transitorio: Tramite la graficazione (*Figura 10*) dell'andamento temporale del limite K , è stato possibile validare visivamente il meccanismo di *Soft Recovery*. L'analisi grafica ha confermato che il limite non subisce oscillazioni caotiche ma, una volta rientrata la crisi, si innalza a gradini sequenziali, confermando la tenuta della doppia soglia logica a protezione delle risorse durante lo svuotamento dell'arretrato.
- Analisi di Sensibilità Parametrica: Il collaudo definitivo del modello è stato condotto variando sistematicamente la capacità fisica del nodo e il limite hardware per osservarne le reazioni stocastiche. Il sistema ha risposto in modo coerente ai principi della Teoria delle Code: l'incremento del gap di server riservati ha prodotto una riduzione monotona e prevedibile della probabilità di violazione dell'Emergenza, validando la correlazione fisica tra risorse esclusive e tutela dello SLA (risultati riportati in *Tabella 12*).

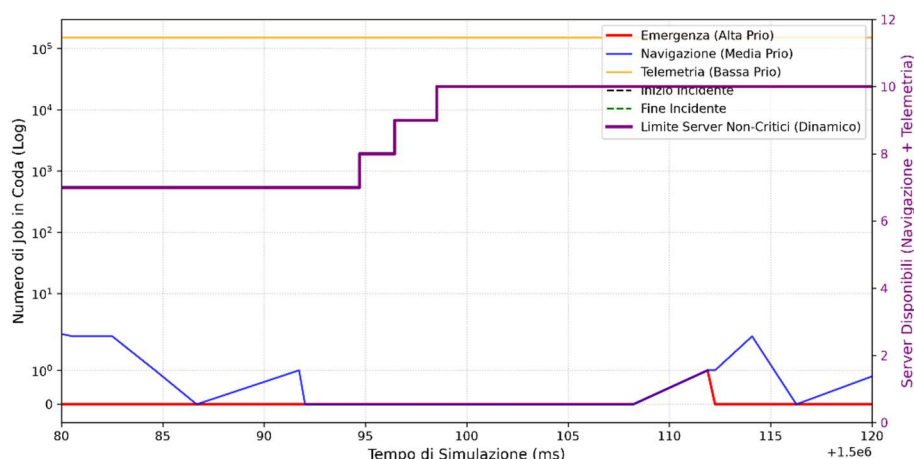


Figura 10: Dinamica transitoria del limite di Trunk Reservation durante lo smaltimento della congestione ($c=10$, $K=7$).

Tabella 12: Impatto del limite K sulle metriche di Emergenza ($c=10$)

Configurazione	Probabilità Media Violazione	Picco Massimo Latenza
$K=9$	$6.2198\% \pm 0.0543\%$	$63.87 \text{ ms} \pm 2.94 \text{ ms}$
$K=8$	$4.0948\% \pm 0.0474\%$	$58.06 \text{ ms} \pm 2.66 \text{ ms}$
$K=7$	$1.5809\% \pm 0.032\%$	$52.53 \text{ ms} \pm 3.47 \text{ ms}$

Progettazione dell'Esperimento

Per valutare quantitativamente l'efficacia della *Adaptive Trunk Reservation*, il nodo Edge è stato sottoposto al medesimo scenario di *stress test* (incidente stradale con tasso di arrivi quadruplicato) analizzato nell'Obiettivo 2. Al fine di garantire un confronto statisticamente inattaccabile, è stata mantenuta la tecnica delle Repliche Indipendenti accoppiata ai Numeri Casuali Comuni (CRN). L'utilizzo degli stessi *seed* di partenza assicura che il sistema affronti l'identica sequenza di traffico, isolando i benefici prestazionali dovuti unicamente all'algoritmo migliorativo. L'esperimento è stato condotto variando in modo combinato il numero totale di server (c) e la soglia di blocco (K).

Presentazione dei Dati

I risultati estratti dalle simulazioni, focalizzati sulle metriche critiche dello SLA di Emergenza e del tempo di recupero del sistema, sono riassunti nella seguente tabella:

Tabella 13: Ricerca della Configurazione Ottimale (Vincolo SLA < 1%)

Server Totali (c)	Limite Background (K)	Gap Riservato	Prob. Violazione SLA	Recovery Time Globale	Esito (< 1%)
11	9	2 server	$2.78 \pm 0.035 \%$	~462.000 ms	FALLITO
12	10	2 server	$1.91 \pm 0.031 \%$	~353.000 ms	FALLITO
10	7	3 server	$1.58 \pm 0.032 \%$	~670.000 ms	FALLITO
11	8	3 server	$0.97 \pm 0.018 \%$	~462.000 ms	SUCCESSO
12	9	3 server	$0.61 \pm 0.016 \%$	~353.000 ms	SUCCESSO

Analisi dei Risultati

L'incrocio dei dati prestazionali rivela pattern fondamentali per determinare il dimensionamento definitivo del nodo Edge:

- L'insufficienza dell'hardware puro: Come base di partenza, i test precedenti avevano dimostrato che l'Offloading Dinamico, da solo, non era in grado di proteggere le Emergenze durante l'incidente. Anche potenziando il sistema a 12 Server in assenza di partizionamento, le violazioni rimanevano inchiodate sopra il 3.80%. Questo conferma che il problema non era la mancanza di risorse, ma l'interferenza causata dalla Non-Preemption.
- La riservazione: I dati in tabella svelano la formula perfetta per la stabilità. Indipendentemente dai server totali, riservare solo 2 server per l'emergenza ($\text{Gap} = 2$) non è sufficiente a scendere sotto l'1% di tolleranza. La svolta avviene impostando un gap di almeno 3 server ($c - K = 3$): con 11 server e limite a 8, le violazioni crollano allo 0.97%, soddisfacendo, seppur di poco, il vincolo contrattuale di progetto.
- Il ruolo della capacità fisica sul Recovery Time: Mentre l'algoritmo di partizionamento governa i tempi di latenza tramite la priorità, la capacità di smaltire la coda arretrata al termine dell'incidente dipende dall'hardware. L'aggiunta di un undicesimo server abbatte il *Recovery Time* dell'intero sistema da circa 668.000 ms (configurazione a 10 server) a circa 462.000 ms, riducendo drasticamente il periodo di sofferenza post-emergenza.

L'efficacia della ripartizione dinamica e il blocco istantaneo delle risorse computazionali sono illustrati visivamente nella *Figura 11*, dove si evince il taglio netto del limite K (linea viola) in esatta corrispondenza della finestra di crisi.

Conclusioni

Sulla base dell'analisi sperimentale, il dimensionamento hardware del nodo Edge equipaggiato con algoritmo di *Adaptive Trunk Reservation* prospetta due validi scenari decisionali, dipendenti dalla tolleranza al rischio e dal budget infrastrutturale. L'architettura a 11 Server (impostata con un limite dinamico $K = 8$) rispetta formalmente il vincolo stringente di progetto, registrando una probabilità di violazione dello 0.97%. Essa rappresenta il miglior compromesso tecnico-economico per minimizzare le risorse fisiche. Tuttavia, operando così a

ridosso della soglia massima tollerata dell'1%, il sistema rimane esposto a fisiologiche fluttuazioni stocastiche in caso di imprevisti estremi.

Pertanto, la configurazione a 12 Server (con soglia $K = 9$) si configura come la scelta raccomandata: abbassando la probabilità di violazione allo 0.61%, fornisce al sistema un margine di sicurezza per i servizi critici, dimezza i tempi di ripristino post-incidente e giustifica l'investimento infrastrutturale nella singola unità computazionale aggiuntiva.

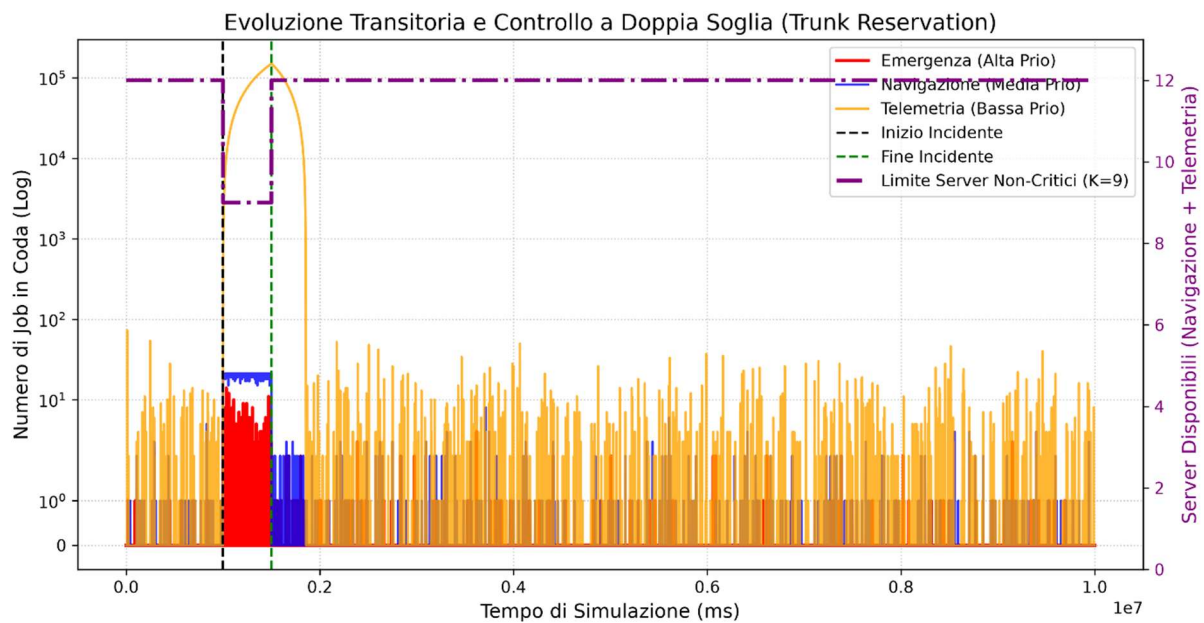


Figura 11: *Evoluzione transitoria delle code con logica di Adaptive Trunk Reservation (Scala Logaritmica, $c=12$). L'asse secondario di destra (linea viola tratteggiata) mostra l'intervento dinamico del limite K imposto ai server di background. All'innesco della Crisis Mode, la soglia crolla istantaneamente a 9, garantendo un gap di 3 server dedicati in via esclusiva alle Emergenze. Al termine dell'emergenza fisica, il blocco viene rimosso e il limite viene ripristinato alla capacità totale del nodo (12 server).*

7. Conclusioni e Sviluppi Futuri

Questo studio ha condotto un'analisi prestazionale completa di un nodo Edge operante in un ambiente Smart City (V2X). Attraverso la modellazione e l'inferenza statistica, il progetto ha permesso di trarre tre conclusioni ingegneristiche fondamentali:

- Obiettivo 1 (Dimensionamento Nominale): È stato dimostrato che la semplice stabilità matematica (code che non divergono) non è sufficiente a garantire la Qualità del Servizio (QoS). Nello scenario nominale, sebbene 9 server garantissero stabilità, è stata necessaria l'allocazione di 10 server per mantenere le violazioni di latenza delle Emergenze strettamente al di sotto del vincolo di progetto ($SLA < 1\%$).
- Obiettivo 2 (Congestione Dinamica): Lo stress test (incidente stradale) ha evidenziato i limiti della priorità *Non-Preemptive*. In caso di crisi, l'Offloading Dinamico a soglia (50 ms) si è rivelato un "salvavita" essenziale per proteggere la memoria del nodo (evitando il *Buffer Overflow* dei pacchetti di Navigazione), ma da solo non è bastato a tutelare le Emergenze, le cui violazioni sono balzate oltre il 7% a causa dei server tenuti in ostaggio dal traffico di background.
- Evoluzione (Trunk Reservation): Il problema della latenza in emergenza è stato risolto introducendo un algoritmo di *Adaptive Trunk Reservation*. Bloccando l'accesso al traffico non critico e garantendo un *gap* di almeno 3 server fisicamente riservati all'Emergenza durante la crisi (configurazione ottima: 12 server totali, limite di background a 9), il nodo è riuscito a ripristinare il rispetto dello SLA (violazioni allo 0.61%) e a dimezzare i tempi di smaltimento dell'arretrato.

Sviluppi Futuri: Il Limite Fisico dell'Offloading

Nonostante il successo dell'algoritmo migliorativo, l'analisi ha messo in luce una criticità intrinseca che apre le porte a futuri sviluppi. Anche nello scenario ottimizzato, il nodo Edge è costretto a deviare oltre 350.000 richieste di Navigazione (circa il 35% del traffico) per sopravvivere all'incidente.

Questo massiccio tasso di deviazione non rappresenta un limite algoritmico, ma un vincolo fisico invalicabile dettato dalle leggi della Teoria delle Code. Durante la crisi, l'intensità di traffico balza a 2 richieste al millisecondo, di cui la metà è dedicata alla Navigazione. Con un tempo di servizio medio di 30 ms, il carico offerto dalla sola Navigazione si attesta a 30 Erlang. Per smaltire tale ondata senza code servirebbero 30 server dedicati unicamente a questa classe,

oltre ad almeno 4 server aggiuntivi per reggere il carico delle Emergenze (3.5 Erlang). Il sistema a 12 server risulta pertanto sotto-dimensionato del 60% rispetto alla mole d'urto dell'incidente, rendendo la deviazione dei pacchetti un obbligo per prevenire il collasso del nodo.

Alla luce di questi dati, i futuri sviluppi del modello di simulazione dovranno abbandonare l'ipotesi irrealistica di un nodo adiacente a "capacità infinita". Un incidente reale blocca un'intera area geografica, e deviare localmente 350.000 pacchetti innescerebbe un collasso a catena (effetto valanga) sull'intera rete Edge limitrofa. Le future indagini si concentreranno quindi su:

- Architetture Fog-to-Cloud Routing: Modificare la logica di offloading orizzontale (Edge-to-Edge) a favore di un instradamento verticale verso il Cloud Centrale. Accettare una latenza superiore (es. 100-200 ms) per il ricalcolo dei percorsi risulta un compromesso accettabile che preserverebbe l'infrastruttura locale.
- Scaling Dinamico delle Risorse: Implementare logiche di auto-scaling (Massive Overprovisioning on-demand) per accendere server dormienti solo durante le crisi, avendo però cura di modellare e mitigare all'interno del simulatore i ritardi intrinseci dovuti ai tempi di accensione fisica delle macchine (*Cold Start*).